

第5章

双(公)钥密码体制

双钥(公钥)体制于1976年由W. Diffie和M. Hellman提出,同时R. Merkle也独立提出了这一体制。J. H. Ellis的文章阐述了公钥密码体制的发明史,说明了CESG的研究人员对双钥密码体制发明所做出的重要贡献。这一体制的最大特点是采用两个密钥将加密和解密能力分开:一个密钥公开作为加密密钥,称为公钥;一个密钥为用户专用,作为解密密钥,称为私钥。通信双方无须事先交换密钥就可进行保密通信。但是从公开的公钥或密文分析出明文或私钥,则在计算上是不可行的。若以公开钥作为加密密钥,以用户专用钥作为解密密钥,则可实现多个用户加密的消息只能由一个用户解读;反之,以用户专用钥作为加密密钥而以公开钥作为解密密钥,则可实现由一个用户加密的消息而使多个用户解读。前者可用于保密通信,后者可用于数字签名。这一体制的出现是密码学史上划时代的事件,它为解决计算机信息网中的安全提供了新的理论和技术基础。

自1976年以来,双钥体制有了飞速发展,人们不仅提出了多种算法,而且出现了不少安全产品,有些已用于NII和GII之中。本章介绍其中的一些主要体制,特别是那些既有安全性,又有实用价值的算法。其中,包括可用于密钥分配、加解密或数字签名的双钥算法。一个好的系统不仅算法要好,还要求能与其他部分(如协议等)进行有机组合。

由于双钥体制的加密变换是公开的,任何人都可以采用选择明文来攻击双钥体制,因此,明文空间必须足够大才能防止穷尽搜索明文空间攻击。这在双钥体制应用中特别重要(如用双钥体制加密会话密钥时,会话密钥要足够长)。一种更强有力的攻击法是选择密文攻击,攻击者选择密文,然后通过某种途径得到相应的明文,多数双钥体制对于选择密文攻击特别敏感。攻击者通常采用两类选择密文攻击:

(1) 冷漠选择密文攻击。在接收到待攻击的密文之前,可以向攻击者提供他们所选择的密文的解密结果。

(2) 自适应选择密文攻击。攻击者可能利用(或接入)被攻击者的解密机(但不知其秘密钥),而可以对他所选择的、与密文有关的待攻击的密文,以及以前询问得到的密文进行解密。

本章介绍双钥体制的基本原理和几种重要算法,如RSA、ElGamal、椭圆曲线、基于身份的密码体制和中国商用密码SM2算法等密码算法。

Diffie [Diffie 1992]曾对双钥体制的发展做了全面论述。

5.1

双钥密码体制的基本概念

对于双钥密码体制来说,其安全性主要取决于构造双钥算法所依赖的数学问题。要求加密函数具有单向性,即求逆的困难性。因此,设计双钥体制的关键是首先要寻求一个合适的单向函数。

5.1.1 单向函数

定义 5-1 令函数 f 是集 A 到集 B 的映射,用 $f:A \rightarrow B$ 表示。若对任意 $x_1 \neq x_2$, $x_1, x_2 \in A$, 有 $f(x_1) \neq f(x_2)$, 则称 f 为单射,或 1-1 映射,或可逆的函数。

f 为可逆的充要条件是,存在函数 $g:B \rightarrow A$, 使对所有 $x \in A$ 有 $g[f(x)] = x$ 。

定义 5-2 一个可逆函数 $f:A \rightarrow B$, 若它满足:

(1) 对所有 $x \in A$, 易于计算 $f(x)$ 。

(2) 对“几乎所有 $x \in A$ ”由 $f(x)$ 求 x “极为困难”,以至于实际上不可能做到,则称 f 为单向 (One-Way) 函数。

定义中的“极为困难”是对现有的计算资源和算法而言。Massey 称此为视在困难性 (Apparent Difficulty), 相应函数称为视在单向函数,以此来与本质上的困难性 (Essential Difficulty) 相区分[Massey 1985]。

例 5-1 令 f 是在有限域 $GF(p)$ 中的指数函数,其中 p 是大素数,即

$$y = f(x) = \alpha^x \quad (5-1)$$

式中, $x \in GF(p)$, x 为满足 $0 \leq x < p-1$ 的整数,其逆运算是 $GF(p)$ 中定义的对数运算,即

$$x = \log_{\alpha} \alpha^x \quad 0 \leq x < p-1 \quad (5-2)$$

显然,由 x 求 y 是容易的,即使当 p 很大,例如 $p \approx 2^{100}$ 时也不难实现。为方便计算,以下令 $\alpha=2$ 。所需的计算量为 $\log$ 次乘法,存储量为 $(\log p)^2 b$, 例如 $p=2^{100}$ 时,需做 100 次乘法。利用高速计算机由 x 计算 α^x 可在 0.1ms 内完成。但是相对于当前计算 $GF(p)$ 中对数最好的算法,要从 α^x 计算 x 所需的存储量大约为 $(3/2) \times \sqrt{p} \log p b$, 运算量大约为 $(1/2) \times \sqrt{p} \log p$ 。当 $p=2^{100}$ 时,所需的计算量为 $(1/2) \times 2^{50} \times 100 \approx 10^{16.7}$ 次,用计算指数一样快的计算机进行计算需时约 $10^{10.7}$ 秒 (1 年 $= 10^{7.5}$ 秒,故约为 1600 年。其中假定存储量的要求能够满足)。由此可见,当 p 很大时, $GF(p)$ 中的 $f(x) = \alpha^x$, $x < p-1$ 为单向函数。

Pohlig 和 Hellman 对 $(p-1)$ 无大素因子时给出一种快速求对数的算法[Pohlig 等 1978]。特别是当 $p=2^n+1$ 时,从 α^x 求 x 的计算量仅需 $(\log p)^2$ 次乘法。对于 $p=2^{160}+1$, 在高速计算机上大约仅需时 10ms。因此,在这种情况下, $f(x) = \alpha^x$ 就不能被认为是单向函数。

综上所述,当对素数 p , 且 $p-1$ 有大的素因子时, $GF(p)$ 上的函数 $f(x) = \alpha^x$ 是一个视在单向函数。寻求在 $GF(p)$ 上求对数的一般快速算法是当前密码学研究中的一个重要课题。

5.1.2 陷门单向函数

单向函数是求逆困难的函数，而陷门单向函数（Trapdoor One-Way Function）是在不知陷门信息下求逆困难的函数，当知道陷门信息后，求逆易于实现。这是 Diffie 和 Hellman[Diffie 等 1976]引入的有用概念。

号码锁在不知预设号码时很难打开，但若知道所设号码则容易开启。太平门是另一例，从里面向外出容易，若无钥匙者反向难进。但如何给陷门单向函数下定义则很棘手，因为：

(1) 陷门函数其实不是单向函数，因为单向函数是在任何条件下求逆都是困难的。

(2) 陷门可能不止一个，通过试验，一个个陷门就可容易地找到逆。如果陷门信息的保密性不强，求逆也就不难。

定义 5-3 陷门单向函数是一类满足下述条件的单向函数： $f_z: A_z \rightarrow B_z$ ， $z \in Z$ ， Z 是陷门信息集。

(1) 对所有 $z \in Z$ ，在给定 z 下容易找到一对算法 E_z 和 D_z ，使对所有 $x \in A$ ，易于计算 f_z 及其逆，即

$$f_z(x) = E_z(x) \quad (5-3)$$

$$D_z(f_z(x)) = x \quad (5-4)$$

而且当给定 z 后容易找到一种算法 F_z ，称 F_z 为可用消息集鉴别函数，对所有 $x \in A$ 易于检验是否 $x \in A_z$ ($A_z \subset A$)， A_z 是可用的明文集。

(2) 对“几乎所有” $z \in Z$ ，当只给定 E_z 和 F_z 时，对“几乎所有” $x \in A_z$ ，“很难”（即“实际上不可能”）从 $y = F_z(x)$ 算出 x 。

(3) 对任一 z ，集 A_z 必须是保密系统中明文集的一个“方便”集。即便于实现明文到它的映射（在双钥密码体制中是默认的条件）。(Diffie 和 Hellman 定义的陷门函数中， $A_z = A$ ，对所有 Z 成立。实际中的 A_z 取决于 Z)。

5.1.3 公钥系统

在一个公钥系统中，所有用户共同选定一个陷门单向函数，加密运算 E 及可用消息集鉴别函数 F 。用户 i 从陷门集中选定 z_i ，并公开 E_{z_i} 和 F_{z_i} 。任一要向用户 i 发送机密消息者，可用 F_{z_i} 检验消息 x 是否在许用消息集之中，然后送 $y = E_{z_i}$ 给用户 x 即可。

在仅知 y ， E_{z_i} 和 F_{z_i} 的情况下，任一用户不能得到 x 。但用户 i 利用陷门信息 z_i ，易于得到 $D_{z_i}(y) = x$ 。

定义 5-4 对 $z \in Z$ 和任意 $x \in X$ ， $F_i(x) \rightarrow y \in Y = X$ 。若

$$F_j(F_i(x)) = F_i(F_j(x)) \quad (5-5)$$

成立，则称 F 为可换单向函数。

可换单向函数在密码学中更有用。

5.1.4 用于构造双钥密码的单向函数

Diffie 和 Hellman 在 1976 年发表的文章虽未给出陷门单向函数，但大大推动了这

方面的研究工作。双钥密码体制的研究在于,给出这种函数的构造方法以及它们的安全性。

陷门单向函数的定义并没有指出这类函数是否存在,但其中指出:一个单钥密码体制,如果能抗击选择明文攻击,就可规定一个陷门单向函数。以其密钥作为陷门信息,则相应的加密函数就是这类函数。这是构造双钥体制的途径。

下面是一些单向函数的例子。目前多数双钥体制是基于这些问题构造的。

1. 多项式求根

有限域 $GF(p)$ 上的一个多项式

$$y = f(x) = x^n + a_{n-1}x^{n-1} + \cdots + a_1x + a_0 \bmod p$$

当给定 $a_0, a_1, \dots, a_{n-1}$, p 及 x 时,很容易求 y , 利用 Honer's 法则,即

$$f(x) = (((\cdots (x + a_{n-1})x + a_{n-2})x + a_{n-3})x + \cdots + a_1)x + a_0 \quad (5-6)$$

最多有 n 次乘法和 $n-1$ 次加法。反之,已知 $y, a_0, \dots, a_{n-1}$, 要求解 x 需能对高次方程求根。这至少要 $\lfloor n^2(lbp)^2 \rfloor$ 次乘法(这里, $\lfloor a \rfloor$ 表示不大于 a 的最大整数), 当 n, p 很大时很难求解。

2. 离散对数 DL (Discrete Logarithm)

给定一大素数 p , $p-1$ 含另一大素数因子 q , 可构造一乘群 Z_p^* , 它是一个 $p-1$ 阶循环群。其生成元为整数 g , $1 < g < p-1$ 。已知 x , 容易求 $y = g^x \bmod p$, 这只需 $\lfloor \lg x \rfloor - 1$ 次乘法, 如 $x = 15 = 1111_2$, $g^{15} = (((1 \cdot g)^2 \cdot g)^2 \cdot g)^2 \cdot g \bmod p$, 要用 $3+4-1=6$ 次乘法。

若已知 y, g, p , 求 $x = \log_g y \bmod p$ 为离散对数问题。最快求解法运算次数渐近值为

$$L(p) = O(\exp\{(1+o(1))\sqrt{\ln p \ln(\ln p)}\}) \quad (5-7)$$

$p=512$ 时, $L(p)=2^{256}=10^{77}$ 。

若离散对数定义在 $GF(2^n)$ 中的 2^n-1 阶循环群上, Shanks 和 Pohlig-Hellman 等的离散对数算法预计算量的渐近式为

$$O(\exp\{(1.405 + o(1))n^{1/3}(\ln n)^{2/3}\}) \quad (5-8)$$

求一特定离散对数的计算量的渐近式为

$$L(p) = O(\exp\{(1.098 + o(1))n^{1/3}(\ln n)^{2/3}\}) \quad (5-9)$$

具体请参阅[LaMacchia 等 1991; McCurley 1990]。

广义离散对数问题是在 n 阶有限循环群 G 上定义的。

3. 大整数分解 FAC (Factorization Problem)

判断一个大奇数 n 是否为素数的有效算法, 大约需要的计算量是 $\lfloor \lg n \rfloor^4$, 当 n 为 256 或 512 位的二元数时, 用当前计算机做可在 10 分钟内完成。

若已知两个大素数 p 和 q , 求 $n = p \cdot q$ 只需一次乘法, 但若由 n , 求 p 和 q , 则是几千年来数论专家的攻关对象。迄今为止, 已知的各种算法的渐近运行时间如下:

(1) 试除法: 最早的也是最慢的算法, 需试验所有小于 $\sqrt{n}$ 的素数, 运行时间为指数函数。

(2) 二次筛 (QS):

$$T(n) = O(\exp\{(1+o(1))\sqrt{\ln n \ln(\ln n)}\}) \quad (5-10)$$

该算法为小于 110 位整数最快的算法, 倍多项式二次筛 (MPQS) 是 QS 算法的变型, 它比 QS 算法更快。MPQS 的双倍大指数变型还要更快一些。

(3) 椭圆曲线 (EC):

$$T(n) = O(\exp\{(1+o(1))\sqrt{2 \ln p \ln(\ln p)}\}) \quad (5-11)$$

(4) 数域筛 (NFS):

$$T(n) = O(\exp\{(1.92 + o(1))(\ln n)^{1/3} (\ln(\ln n))^{2/3}\}) \quad (5-12)$$

式中, p 是 n 的最小的素因子, 最坏的情况下 $p \approx n^{1/2}$ 。当 $n \approx 2^{664}$, 要用 3.8×10^9 年 (一秒进行 100 万次运算)。虽然整数分解问题已进行了很长时间研究, 但至今尚未发现快速算法。目前对于大于 110 位的整数数域筛是最快的算法, 曾用于分解第 9 个 Fermat 数。目前的进展主要是靠计算机资源来实现的。二次筛法可参阅[Pomerance 1984; Carton 等 1988]; 数域筛法可参阅[Lenstra 等 1993]; 椭圆曲线法参阅[Pollard 1993; Lenstra 1987; Montgomery 1987]。

$T(n)$ 与 $L(p)$ 的表示式大致相同, 一般当 $n=p$ 时, 解离散对数要更难些。

RSA 问题是 FAC 问题的一个特例。 n 是两个素数 p 和 q 之积, 给定 n 后求素因子 p 和 q 的问题称为 RSAP。求 $n = pq$ 分解问题有以下几种形式:

- (1) 分解整数 n 为 p 和 q 。
- (2) 给定整数 M 和 C , 求 d 使 $C^d \equiv M \pmod{n}$ 。
- (3) 给定整数 e 和 C , 求 M 使 $M^e \equiv C \pmod{n}$ 。
- (4) 给定整数 x 和 C , 决定是否存在整数 y 使 $x \equiv y^2 \pmod{n}$ (二次剩余问题)。

4. Diffie-Hellman 问题 (DHP)

给定素数 p , 令 α 为 Z_p^* 的生成元, 若已知 α^a 和 α^b , 求 α^{ab} 的问题为 Diffie-Hellman 问题, 简称 DHP。若 α 为循环群 G 的生成元, 且已知 α^a 和 α^b 为 G 中的元素, 求 α^{ab} 的问题为广义 Diffie-Hellman 问题, 简记为 GDHP[den Boer 1988; Maurer 1994b; Waldvogel 等 1993; McCurley 1988]。

在[Menezes 等 1997]一书的第 4 章对双钥密码体制公钥参数的生成和有关算法进行了全面介绍, 该书的第 3 章对密码中用到的数学难题进行了全面系统的论述。此外, 还可参阅[Pomerance 1990; Adleman 等 1994; Bach 1990; Lenstra 等 1990a, 1990b]。

5.2

RSA 密码体制

1978 年, MIT 的 3 位年轻数学家 R.L.Rivest, A.Shamir 和 L.Adleman 发现了一种用数论构造双钥的方法[Rivest 等 1978, 1979], 称为 MIT 体制, 后来被广泛称为 RSA 体制。它既可用于加密, 又可用于数字签名, 易懂且易于实现, 是目前仍然安全并且逐步

被广泛应用的一种体制。国际上一些标准化组织(如 ISO, ITU 和 SWIFT 等)均已接受 RSA 体制作为标准。在因特网中所采用的 PGP (Pretty Good Privacy) 中也将 RSA 作为传送会话密钥和数字签名的标准算法。

RSA 算法的安全性基于 5.1 节介绍的数论中大整数分解的困难性。

5.2.1 RSA 密码体制

独立选取两个大素数 p_1 和 p_2 (各 100~200 位十进制数字), 计算

$$n = p_1 \times p_2 \quad (5-13)$$

其欧拉函数值为

$$\varphi(n) = (p_1 - 1)(p_2 - 1) \quad (5-14)$$

随机选一整数 e , $1 \leq e < \varphi(n)$, $(\varphi(n), e) = 1$ 。因而在模 $\varphi(n)$ 下, e 有逆元

$$d = e^{-1} \bmod \varphi(n) \quad (5-15)$$

取公钥为 n , e 。密钥为 d (p_1 , p_2 不再需要, 可以销毁)。

加密: 将明文分组, 各组在 $\bmod n$ 下, 可唯一地表示出来(以二元数字表示, 选 2 的最大幂小于 n)。各组长达 200 位十进制数字。可用明文集为

$$A_z = \{x : 1 \leq x < n, (x, n) = 1\}$$

注意, $(x, n) \neq 1$ 是很危险的。 $x \in A_z$ 的概率

$$\frac{\varphi(n)}{n} = \frac{(p_1 - 1)(p_2 - 1)}{p_1 p_2} = 1 - \frac{1}{p_1} - \frac{1}{p_2} + \frac{1}{p_1 p_2} \rightarrow 1$$

密文

$$y = x^e \bmod n \quad (5-16)$$

解密:

$$x = y^d \bmod n \quad (5-17)$$

证明: $y^d = (x^e)^d = x^{de}$, 因为 $de \equiv 1 \bmod \varphi(n)$ 而有 $de = q\varphi(n) + 1$ 。由欧拉定理, $(x, n) = 1$ 意味 $x^{\varphi(n)} \equiv 1 \bmod n$, 故有

$$y^d = x^{de} = x^{q\varphi(n)+1} = x \cdot x^{q\varphi(n)} = x \cdot 1 = x \bmod n$$

陷门函数: $Z = (p_1, p_2, d)$

例 5-2 选 $p_1 = 47$, $p_2 = 71$, 则 $n = 47 \times 71 = 3337$, $\varphi(n) = 46 \times 70 = 3220$ 。若选 $e = 79$, 可计算 $d = e^{-1} \bmod 3220 = 1019$ 。公开 $n = 3337$ 和 $e = 79$ 。密钥 $d = 1019$ 。销毁 p_1 , p_2 。

令 $x = 688 \ 232 \ 687 \ 966 \ 668 \ 3$, 分组得 $x_1 = 688$, $x_2 = 232$, $x_3 = 687$, $x_4 = 966$, $x_5 = 668$, $x_6 = 3$ 。 x_1 的加密为 $(688)^{79} \bmod 3337 = 1570 = y_1$ 。同样, 可计算出其他各组密文。得到密文 $y = 1570 \ 2756 \ 2714 \ 2423 \ 158$ 。

第一组密文的解密为 $(1570)^{1019} \bmod 3337 = 688 = x_1$ 。类似地可解出其他各组密文。

RSA 加密实质上是一种 $Z_n \rightarrow Z_n$ 上的单表代换。给定 $n = p_1 p_2$ 和合法明文 $x \in Z_n$, 其相应密文 $y = x^e \bmod n \in Z_n$ 。对于 $x \neq x'$, 必有 $y \neq y'$ 。 Z_n 中的任一元素 ($0, p_1, p_2$ 除外) 是一个明文, 但它也是与某个明文相对应的一个密文。因此, RSA 是 $Z_n \rightarrow Z_n$ 的一种单表代换密码, 关键在于 n 极大时在不知道陷门信息下极难确定这种对应关系, 而用模指

数算法又易于实现一种给定的代换。正是因为这种对应性，使 RSA 不仅可以用于加密，也可以用于数字签名。

5.2.2 RSA 的安全性

1. 分解模数 n

在理论上，RSA 的安全性取决于模 n 分解的困难性，但数学上至今还未证明分解模就是攻击 RSA 的最佳方法，也未证明分解大整数就是 NP 问题，可能有尚未发现的多项式时间分解算法。人们完全可以设想有另外的途径破译 RSA，如求解密指数 d 或找到 $(p_1 - 1)(p_2 - 1)$ 等。但这些途径都不比分解 n 更容易。甚至有文献[Alexi 等 1988]曾揭示，从 RSA 加密的密文恢复某些位的困难性也和恢复整组明文一样困难。

当前的技术进展使分解算法和计算能力在不断提高，计算所需的硬件费用在不断下降。110 位十进制数字早已能分解。Rivest 等最初悬赏\$100 的 RSA-129，已由包括 5 大洲 43 个国家六百多人参加。人们通过 Internet，用 1600 台计算机同时产生 820 条指令数据，耗时 8 个月，于 1994 年 4 月 2 日利用二次筛法分解出为 64 位和 65 位的两个因子，原来估计要用 4 亿亿年。所给密文的译文为“这些魔文是容易受惊的鱼鹰”。这是有史以来最大规模的数学运算。RSA-130 于 1996 年 4 月 10 日利用数域筛法分解出来，目前正在向更大的数，特别是 512b RSA，即 RSA-154 冲击[Cowie 等 1996]。表 5-1 给出了采用广义数域筛分解不同长度 RSA 公钥模所需的计算机资源。

表 5-1 采用广义域筛所需计算机资源

密钥长 (b)	所需的 MIPS-年*
116 (Blacknet 密钥)	400
129	5 000
512	30 000
768	200 000 000
1024	300 000 000 000
2048	300 000 000 000 000 000 000

*: MIPS-年指以每秒执行 1 000 000 条指令的计算机运行一年。

表 5-2 为采用 NSF 算法破译 RSA 体制与用穷搜索密钥法破译单钥体制的等价密钥长度。

表 5-2 等价密钥长度

单 钥 体 制	RSA 体制	单 钥 体 制	RSA 体制
56b	384b	112b	1792b
64b	512b	128b	2304b
80b	768b		

因此，如果要用 RSA，需要采用足够大的整数。512b (154 位)、664b (200 位) 已有实用产品。也有人想用 1024b 的模。若以每秒可进行 100 万步的计算资源分解 664b

大整数, 需要完成 10^{23} 步, 即要用 1000 年。在 European Institute for System Security Workshop 上, 与会者认为 1024b 模在今后 10 年内足够安全。Simmons 预测 150 位数将在 21 世纪被分解。数学家估计分解 $x+10$ 位数的困难程度约为分解 x 的 10 倍。目前, 512b 模 (约 155 位) 在短期内仍十分安全, 但大素数分解工作在 WWW 上的大协作已构成对 512b 模 RSA 的严重威胁, 很快可能要采用 768b 甚至 1024b 的模。

大整数分解算法的研究是当前数论和密码理论研究的一个重要课题, 可参阅相关文献[Adleman 1991; Bressoud 1989; Buhler 等 1993; Coppersmith 1993; Denny 等 1994; Dobbertin 1996; Lenstra 1987; Montgomery 1987; Pomerance 1990, 1994; Silverman 1987; van Oorschot 1992]。

2. 其他途径

从 n 若能求出 $\varphi(n)$, 则可求得 p_1, p_2 , 因为

$$n - \varphi(n) + 1 = p_1 p_2 - (p_1 - 1)(p_2 - 1) + 1 = p_1 + p_2$$

而

$$\sqrt{n^2 - 4n} = p_1 - p_2$$

但已经证明, 求 $\varphi(n)$ 的困难性等价于分解 n 的困难性。

从 n 求 d 也等价于分解 n 。

目前尚不知道是否存在一种无须借助于分解 n 的攻击法, 也未能证明破译 RSA 的任何方法都等价于大整数分解问题。

3. 迭代攻击法

Simmons 和 Norris 曾提出迭代或循环攻击法。例如, 给定一 RSA 的参数为 $(n, e, y) = (35, 17, 3)$, 可由 $y_0 = y = 3$ 计算 $y_1 = 3^{17} = 33 \bmod 35$ 。再由 y_1 计算 $y_2 = y_1^{17} = 3 \bmod 35$, 从而得到明文 $x = y_1 = 33 \bmod 35$ 。一般对明文 x 加密多次, 直到再现 x 为止。Rivest 证明[Rivest 1978], 当 $p_1 - 1$ 和 $p_2 - 1$ 中含有大素数因子, 且 n 足够大时, 这种攻击法成功的概率趋于 0。

4. 选择密文攻击

(1) 消息破译。攻击者收集用户 A 以公钥 e 加密的密文 $y = x^e \bmod n$, 并想分析出明文 x 。选随机数 $r < n$, 计算 $y_1 = r^e \bmod n$, 这意味 $r = y_1^d \bmod n$ 。计算 $y_2 = y_1 \times y \bmod n$ 。令 $t = r^{-1} \bmod n$, 则 $t = y_1^{-d} \bmod n$ 。

如果攻击者请 A 对消息 y_2 进行解密, 得到 $s = y_2^d \bmod n$ 。攻击者计算 $ts \bmod n = y_1^{-d} \times y_2^d \bmod n = y_1^{-d} \times y_1^d \times y^d \bmod n = y^d \bmod n = x$, 得到了明文。

(2) 骗取仲裁签名。在有仲裁情况下, A 有一个文件要求仲裁, 可先将其送给仲裁 T , T 以 RSA 的密钥进行签署后回送给 A (未用单向 Hash 函数, 只以密钥对整个消息加密)。

攻击者有一个消息要 T 签署, 但 T 并不情愿给他签, 因为该消息可能有伪造的时戳, 也可能是来自其他人的消息。但攻击者可用下述方法骗取 T 的签名。令攻击者的消息为 x , 他首先任意选一个数 N , 计算 $y = N^e \bmod n$ (e 是 T 的公钥), 然后计算 $M = yx$, 送给

T , T 将签名的结果 $M^d \bmod n$ 送给攻击者, 则有 $(M^d \bmod n)N^{-1} \bmod n = (yx)^d \cdot N^{-1} \bmod n = x^d y^d \cdot N^{-1} \bmod n = x^d NN^{-1} \bmod n = x^d \bmod n$, 此为 T 对 x 的签名。

所以能有这类攻击是因为指数运算保持了输入的乘法结构。

(3) 骗取用户签名。攻击者可构造两条消息 x_1 和 x_2 , 凑出所要的 $x_3 \equiv x_1 \times x_2 \bmod n$ 。

首先他可得到用户 A 对 x_1 和 x_2 的签名 $x_1^d \bmod n$ 和 $x_2^d \bmod n$, 则可计算 $x_3^d \bmod n \equiv (x_1^d \bmod n) \cdot (x_2^d \bmod n) \bmod n$ 。

因此, 任何时候不要为不相识的人签署随机性文件, 最好先采用单向 Hash 函数。ISO 9796 的分组格式可以防止这类攻击。

有关选择密文攻击 RSA 体制的研究可参阅相关文献。

5. 公用模攻击

若很多人共用同一模数 n , 各自选择不同的 e 和 d , 这样实现当然简单, 但是不安全。若明文以两个不同的密钥加密, 在共用同一个模下, 若两个密钥互素 (一般如此), 则可用任一密钥恢复明文[Simmons 1983]。

设 e_1 和 e_2 是两个互素的不同密钥, 共用模为 n , 对同一明文 x 加密得 $y_1 = x^{e_1} \bmod n$, $y_2 = x^{e_2} \bmod n$ 。分析者知道 n, e_1, e_2, y_1 和 y_2 。因为 $(e_1, e_2) = 1$, 所以有 $r \cdot e_1 + s \cdot e_2 = 1$ 。假定 r 为负数, 由 Euclidean 算法可计算

$$(y_1^{-1})^{-r} \cdot y_2^s = x \bmod n$$

还有两种攻击共用模 RSA 的方法:

(1) 用概率方法可分解 n 。

(2) 用确定性算法可计算某一用户密钥而不需要分解 n 。详细内容可参阅[Moore 1988; Simmons 1983]。

6. 低加密指数攻击

采用小的 e 可以加快加密和验证签名的速度, 且所需的存储密钥空间小, 但若加密密钥 e 选择得太小, 则容易受到攻击。

令网络中的 3 个用户的加密密钥 e 均选 3, 而有不同的模 n_1, n_2, n_3 。若一个用户将消息 x 传给 3 个用户的密文分别为

$$y_1 = x^3 \bmod n_1 \quad x < n_1$$

$$y_2 = x^3 \bmod n_2 \quad x < n_2$$

$$y_3 = x^3 \bmod n_3 \quad x < n_3$$

一般选 n_1, n_2, n_3 互素 (否则, 可求出公因子而降低安全性), 利用中国剩余定理, 可从 y_1, y_2, y_3 求出

$$y = x^3 \bmod (n_1 n_2 n_3)$$

由 $x < n_1, x < n_2, x < n_3$, 可得 $x^3 < n_1 \cdot n_2 \cdot n_3$, 故有 $\sqrt[3]{y} = x$ 。

若 x 后加时戳

$$y_1 = (2^t x + t_1)^3 \bmod n_1$$

$$y_2 = (2^t x + t_2)^3 \bmod n_2$$

$$y_3 = (2^t x + t_3)^3 \bmod n_3$$

t 是 t_1, t_2, t_3 的二元表示位数, 可防止这类攻击。Håstad 将上述攻击扩展为 k 个用户, 即将相同的消息 x 传给 k 个人, 只要 $k > e(e+1)/2$, 采用低指数也可有效攻击。因此, 为抗击这种攻击 e 必须选得足够大。一般 e 选为 16 位素数时, 既可兼顾快速加密, 又可防止这类攻击。

对短的消息, 可用随机数字填充, 以防止低加密指数攻击。

d 太小也不行。Wiener 指出, 对 $e < n$, 而 $d < n/4$, 则可以攻破这类 RSA 体制。Coppersmith 对 RSA 的低指数攻击做了进一步研究。

7. 定时攻击法

定时 (Timing) 攻击法由 P. Kocher 提出, 利用测定 RSA 解密所进行的模指数运算的时间来估计解密指数 d , 然后再精确定出 d 的取值。另外还可采用盲化技术, 即首先将数据进行盲化运算, 再进行加密运算, 而后做去盲运算。这样做虽然不能使解密运算时间保持不变, 但计算时间被随机化而难于推测解密所进行的指数运算的时间 [Unruh 1996]。

8. 消息隐匿问题

对明文 x , $0 \leq x \leq n-1$, 采用 RSA 体制加密, 可能出现 $x^e = x \bmod n$, 致使消息暴露。这是明文在 RSA 加密下的不动点。总有一些不动点, 如 $x=0, 1$ 和 $n-1$ 。一般有 $[1+\gcd(e-1, p-1)] \cdot [1+\gcd(e-1, q-1)]$ 个不动点。由于 $e-1, p-1$ 和 $q-1$ 都是偶数, 所以不动点至少为 9 个。一般来说, 不动点个数相当少, 可以忽略不计 [Blakley 等 1979]。

Kaliski 和 Robshaw 曾对 RSA 的安全性进行全面评述。有关 RSA 算法用于认证协议的安全性研究可参阅 [Coppersmith 等 1996; Franklin 等 1995]。

5.2.3 RSA 的参数选择

综上所述, 为了保证 RSA 体制的安全, 必须仔细选择各参数。有关大素数的求法可参阅其他文献。

1. n 的确定

(1) $n = p_1 \times p_2$, p_1 与 p_2 必须为强素数 (Strong Prime)。强素数 p 的条件如下:

- ① 存在两个大素数 p_1 和 p_2 , $p_1 | (p-1)$, $p_2 | (p+1)$ 。
- ② 存在 4 个大素数 r_1, s_1, r_2 及 s_2 , 使 $r_1 | (p_1-1)$, $s_1 | (p_1+1)$, $r_2 | (p_2-1)$, $s_2 | (p_2+1)$ 。称 r_1, r_2, s_1 和 s_2 为三级素数 (Level-3); p_1 和 p_2 为二级素数。

采用强素数的理由如下: 若 $p-1 = \prod_{i=1}^t p_i^{a_i}$, p_i 为素数, a_i 为正整数。分解式中 $p_i < B$,

B 为已知的小整数, 则存在一种 $p-1$ 的分解法, 使得易于分解 n 。令 $n=pq$, 且 $p-1$ 满足上述条件, $p_i < B$ 。令 $a \geq a_i, i=1, 2, \dots, t$ 。即可构造

$$R = \prod_{i=1}^t p_i^{a_i} \quad (5-18)$$

显然 $(p-1) \nmid R$ 。由费尔马定理有 $2^R \equiv 1 \pmod p$ 。令 $2^R = x \pmod n$ 。若 $x=1$ 则选3代2，直到出现 $x \neq 1$ 。此时，由 $\text{GCD}(x-1, n)=p$ ，就得到 n 的分解因子 p 和 q 。

例 5-3 $n=pq=118\,829$ ，选 $B=14$ ， $a_i=1$ ，由加法链算法

$$R = \prod_{p_i < B} p_i = 2 \times 3 \times 5 \times 7 \times 11 \times 13 = 30\,030$$

且 $2^R = 103\,935 \pmod{118\,829}$ 。由欧几里德算法易求 $\text{GCD}(103\,935-1, 118\,829)=331$ ，从而 $n=331 \times 359$ 。这是由于 $331-1=2 \times 3 \times 5 \times 11$ 为小素数因子之积。

Williams 给出类似的 $p+1$ 的分解算法。

(2) p_1 与 p_2 之差要大。若 p_1 与 p_2 之差很小，则可由 $n=p_1 p_2$ 估计 $(p_1+p_2)/2=n^{1/2}$ ，则由 $((p_1+p_2)/2)^2 - n = ((p_1-p_2)/2)^2$ 。上式右边为小的平方数，可以试验给出 p_1, p_2 的值。

例 5-4 $n=164\,009$ ，估计 $(p_1+p_2)/2 \approx 405$ ，由 $405^2 - n = 16 = 4^2$ ，可得 $(p_1+p_2)/2=405$ ， $(p_1-p_2)/2=4$ ， $p_1=409$ ， $p_2=401$ 。

(3) p_1-1 与 p_2-1 的最大公因子要小。在唯密文攻击下，设破译者截获密文 $y=x^e \pmod n$ 。破译者做下述递推计算 (Simmons 等 1977)：

$$y_i = (y_{i-1})^e \pmod n = (x^e)^i \pmod n$$

若 $ei \equiv 1 \pmod{\phi(n)}$ ，则有 $y_i = (x^e)^i = x \pmod n$ 。若 i 小，则由此攻击法易得明文 x 。由 Euler 定理知， $i = \phi((p_1-1)(p_2-1))$ ，若 p_1-1 和 p_2-1 的最大公因子小，则 i 值大，如 $i=(p_1-1)(p_2-1)/2$ ，此攻击法难于奏效。

(4) p_1, p_2 要足够大，以使 n 分解在计算上不可行。近十多年来，大整数分解因子的进展如表 5-3 所示。

表 5-3 大整数分解因子的进展

年 度	分解数（十进制）位数	机 型	时 间
1983	47	HP PC	3 天
1983	69	Cray 大型计算机	32 h
1988	90	25 个 Sun 工作站	数周
1989	95	1 MZP 处理器	1 个月
1989	105	八十多个工作站	数周
1993	110	128×128 处理器 (0.2MIPS)	1 个月
1994	129	1600 部计算机	8 个月

2. e 的选取原则

$(e, \phi(n))=1$ 的条件易于满足，因为两个随机数为互素的概率约为 3/5。 e 小时，加密速度快，有学者 [Knuth 1981a, 1981b; Shamir 1984] 曾建议采用 $e=3$ 。但 e 太小则存在一些问题 [Coppersmith 等 1996]。

(1) e 不可过小。

① 若 e 小， x 小， $y=x^e \pmod n$ ，当 $x^e < n$ ，则未取模，由 y 直接开 e 次方可求 x 。

② 易遭低指数攻击。

(2) 选 e 在 $\text{mod } \phi(n)$ 中的阶数, 即 $i, e^i \equiv 1 \text{ mod } \phi(n)$, i 达到 $(p_1-1)(p_2-1)/2$ 。

3. d 的选择

e 选定后可用 Euclidean 算法在多项式时间内求出 d 。 d 要大于 $n^{1/4}$ 。 d 小, 签名和解密运算快, 这在 IC 卡中尤为重要 (复杂的加密和验证签名可由主机来做)。类似于加密下的情况, d 不能太小, 否则由已知明文攻击, 构造 (迭代地做) $y = x^e \text{ mod } n$, 再猜测 d 值, 做 $x^d \text{ mod } n$, 直到试凑出 $x^d \equiv 1 \text{ mod } n$ 是 d 值就行了。Wiener 给出对小 d 的系统攻击法, 证明了当 d 长度小于 n 的 $1/4$ 时, 由连分式算法, 可在多项式时间内求出 d 值。至于这是否可推广至 $1/2$, 目前还不知道。

5.2.4 RSA 体制应用中的其他问题

(1) 不可用公共模。一个网, 由一个密钥产生中心 (Key Generation Center, KGC) 采用一个公共模, 分发多对密钥, 并公布相应公钥 e_i , 这当然使密钥管理简化, 存储空间小, 且无重新分组 (Reblocking) 问题, 但如前所述, 它在安全上会带来问题。

(2) 明文摘要尽可能得大。明文摘要尽可能得大, 以使在已知密文下, 要猜测明文无异于完全随机等概。Simmons 和 Holdridge 利用先验不等概率性, 攻破一语音加密系统, 明文有 $2^{32} \approx 4.3 \times 10^9$, 但熵值低, 仅为 $16 \sim 18\text{b}$, 用预先选定的 10^5 (约 2^{17}) 明密文对, 将收到的密文与存储的数比较, 符合者则接收, 否则弃之, 并还原录音, 则有 90% 以上的原始语音可还原。

可在明文分组中加上随机乱数得

$$M' = 2^t M + r$$

式中, t 是 r 的二元表示位数。解得 M' 后除去后 t 位乱数 r 即可。

(3) 用于签名时, 要采用 Hash 函数。

5.2.5 RSA 的实现

硬件实现 RSA 的最快速度也仅为 DES 的 $1/1000$, 512b 模下的 VLSI 硬件实现只达 64kb/s。目前计划开发 512b RSA, 达 1Mb/s 的芯片。1024b RSA 加密芯片也在开发中。人们在努力将 RSA 体制用于灵巧卡技术中。有关 RSA 的硬件实现的研制和一些产品, 可参阅 [Schneier 1996]。508b RSA 的硬件实现的速率可达 225kb/s。

软件实现 RSA 的速度只为 DES 的软件实现的 $1/100$, 在速度上 RSA 无法与对称密钥体制相比, 因而 RSA 体制多只用于密钥交换和认证。512b RSA 的软件实现的速率可达 11kb/s。

如果适当选择 RSA 的参数, 可以大大加快速度。例如, 选 e 为 3, 17 或 65 537 ($2^{16}+1$) 的二进制表示式中都只有两个 1, 大大减少了运算量。X. 509 建议用 65 537, PEM 建议用 3 [RFC1423 1993], PKCS#1 建议用 65 537 [RSA Lab 1993]。在消息后填充随机数字时, 不会出现任何安全问题。

中国剩余定理可以用来加速密钥运算 [Rabin 1979]。

5.3

ElGamal 密码体制

ElGamal 密码体制由 ElGamal 提出[ElGamal 1984, 1985], 它是一种基于离散对数问题的双钥密码体制, 既可用于加密, 又可用于签名。有关离散对数的计算可参阅相关文献[Wang 等 1999]。

5.3.1 密钥生成

令 Z_p 是一个有 p 个元素的有限域, p 是一个素数, 令 g 是 Z_p^* (Z_p 中除去 0 元素) 中的一个本原元或其生成元。明文集 M 为 Z_p^* , 密文集 C 为 $Z_p^* \times Z_p^*$ 。

公钥: 选定 g ($g < p$ 的生成元), 计算公钥

$$\beta \equiv g^\alpha \bmod p \quad (5-19)$$

密钥: $\alpha < p$

5.3.2 加解密

选择随机数 $k \in Z_{p-1}$, 且 $(k, p-1) = 1$, 计算:

$$y_1 = g^k \bmod p \quad (\text{随机数 } k \text{ 被加密}) \quad (5-20)$$

$$y_2 = m\beta^k \bmod p \quad (\text{明文被随机数 } k \text{ 和公钥 } \beta \text{ 加密}) \quad (5-21)$$

其中 m 是要发送的明文组。密文由上述两部分 y_1 、 y_2 级联构成, 即密文 $c = y_1 \| y_2$ 。

特点: 密文由明文和所选随机数 k 来定, 因而是非确定性加密, 一般称之为随机化 (Randomized) 加密, 对同一明文由于不同时刻的随机数 k 不同而给出不同的密文。其代价是使数据扩展一倍。

解密: 收到密文组 c 后, 计算

$$m = y_2 / y_1^\alpha = m\beta^k / g^{k\alpha} = mg^{ak} / g^{k\alpha} \bmod p \quad (5-22)$$

例 5-5 选 $p = 2579$, $g = 2$, $\alpha = 765$, 计算出 $\beta = g^{765} \bmod 2579 = 949$ 。若明文组为 $m = 1299$, 今选随机数 $k = 853$, 可算出 $y_1 \equiv 2^{853} \bmod 2579 = 435$ 及 $y_2 \equiv 1299 \times 949^{853} \bmod 2579 = 2396$ 。密文 $c = (435, 2396)$ 。解密时由 c 可算出消息组 $M \equiv 2396 / (435)^{765} \bmod 2579 = 1299$ 。

5.3.3 安全性

本体制基于 Z_p^* 中有限群上的离散对数的困难性。Haber 和 Lenstra 曾指出 $\bmod p$ 生成的离散对数密码可能存在陷门, 有些“弱”素数 p 下的离散对数较容易求解。但文献[Gordon 1992]中已证明, 不难发现这类陷门从而可以避免选用这类素数。

有关随机化加密的统一论述可参阅相关文献。McCurely 将 ElGamal 方案推广到 Z_n^* 上的单元群, 并证明其破译难度至少相当于分解 n , 破译者即使知道了 n 的分解, 也还要解模 n 的因子的 Diffie-Hellman 问题[Menezes 等 1997]。

5.4

椭圆曲线密码体制

椭圆曲线(Elliptic Curve)作为代数几何中的重要问题已有一百多年的研究历史,积累了大量的研究文献,但直到1985年,N. Koblitz和V. Miller才独立将其引入密码学中,成为构造双钥密码体制的一个有力工具[Koblitz 1987, Miller 1985]。利用有限域 $GF(2^m)$ 上的椭圆曲线点集所构成的群上定义的离散对数系统,可以构造出基于有限域上离散对数的一些如Diffie-Hellman, ElGamal, Schnorr, DSA等双钥体制。对这种椭圆曲线离散对数密码体制(ECDLC)安全性的研究已进行了十余年,尚未发现明显的弱点。它有可能以更小规模 of 软、硬件实现有限域上具有相同安全性的同类体制,具体内容可参阅相关文献[Menezes等 1993a; Koblitz 1987; Demtyko 1993; Koyama等 1991]。

目前,大多数使用公钥密码学进行加密和数字签名的产品和标准都使用了RSA算法。为了保证RSA的安全性,近年来所采用的密钥长度不断增加,这直接导致RSA计算量的增加,对于其应用造成影响。最近,椭圆曲线密码(ECC)对RSA的应用提出了巨大挑战。在公钥密码的标准化过程中,IEEE P1363标准已经考虑使用ECC。

与RSA相比,ECC的主要优点是可以使用比RSA更短的密钥获得相同水平的安全性,其计算量大大减少。另一方面,虽然ECC的理论已经成熟,但直到最近才出现这方面的产品,对ECC的密码分析刚刚起步,因此ECC的可信度还有待进一步验证。

ECC比RSA更难描述。关于ECC的完整数学描述已经超出本书的范围。

5.4.1 实数域上的椭圆曲线

椭圆曲线并不是椭圆。之所以称为椭圆曲线,是因为它们与计算椭圆周长的方程相似,也用3次方程来表示。一般来说,椭圆曲线的3次方程形式为

$$y^2 + axy + by = x^3 + cx^2 + dx + e$$

其中, a, b, c, d 和 e 是实数, x 和 y 在实数上取值。事实上,将方程式限制为下述形式就已经足够:

$$y^2 = x^3 + ax + b \quad (5-23)$$

因为方程中的指数最高为3,所以称为3次方程。椭圆曲线的定义中还包含一个无穷远点或叫做零点的元素,记为 O ,这个概念将在后面讨论。为了画出该曲线,需要计算:

$$y = \sqrt{x^3 + ax + b}$$

对于给定的 a 和 b ,以及 x 的每个取值,需画出 y 的正值和负值,这样每一曲线都关于 $y=0$ 对称。图5-1给出了椭圆曲线的两个例子。

从图中可见,椭圆曲线关于 $y=0$ 对称。

现在考虑满足式(5-23)的所有点 (x, y) 和元素 O 所组成的点集 $E(a, b)$ 。 (a, b) 的值不同,则相应的集合 $E(a, b)$ 也不同。图5-1中的两条曲线可以分别用集合 $E(-1, 0)$ 和

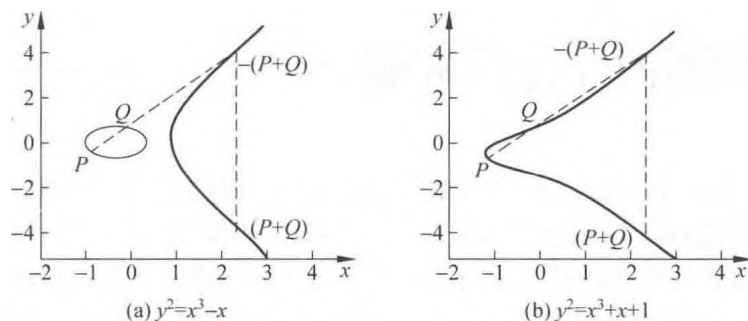


图 5-1 椭圆曲线的两个例子

$E(1, 1)$ 表示。

可以证明：只要 $x^3 + ax + b$ 无重复因子，则可基于集合 $E(a, b)$ 定义一个群。这等价于条件：

$$4a^3 + 27b^2 \neq 0 \quad (5-24)$$

下面在 $E(a, b)$ 上定义加法运算，用“+”表示，其中 a 和 b 满足式 (5-24)。用几何术语可这样定义加法的运算规则：如果椭圆曲线上的 3 个点位于同一直线上，那么它们的和为 O 。进一步可定义椭圆曲线上的加法的运算规则如下：

(1) O 是加法的单位元。这样有 $O = -O$ ；对于椭圆曲线上的任意一点 P ，有 $P + O = P$ 。

(2) 设 $P_1 = (x, y)$ 是椭圆曲线上的点 (图 5-1)，它的加法逆元定义为 $P_2 = -P_1 = (x, -y)$ 。

这是因为 P_1 和 P_2 的连线延长到无穷远时，得到椭圆曲线上的另一点 O ，即椭圆曲线上的 3 点 P_1 、 P_2 和 O 共线，所以 $P_1 + P_2 + O = O$ ， $P_1 + P_2 = O$ ，即 $P_2 = -P_1$ 。

(3) 设 Q 和 R 是椭圆曲线上 x 坐标不同的两点， $Q + R$ 的定义如下：画一条通过 Q 和 R 的直线与椭圆曲线交于 P_1 （这一交点是唯一的，除非所做的直线是 Q 点或 R 点的切线，此时分别取 $P_1 = Q$ 和 $P_1 = R$ ）。由 $Q + R + P_1 = O$ 得 $Q + R = -P_1$ 。

(4) 点 Q 的倍数定义如下：在 Q 点做椭圆曲线的一条切线，设切线与椭圆曲线交于点 S ，定义 $2Q = Q + Q = -S$ 。类似地，可以定义 $3Q = Q + Q + Q = \dots$ ，等等。

以上定义的加法具有加法运算的一般性质，如交换律和结合律等。

5.4.2 有限域 Z_p 上的椭圆曲线

椭圆曲线密码体制使用的是变元和系数均为有限域中元素的椭圆曲线。密码应用中所使用的两类椭圆曲线是定义在有限域 Z_p 上的素曲线 (Prime Curves) 和在 $GF(2^m)$ 上构造的二元曲线。文献[Fernandes 1999]指出，因为不需要二元曲线所要求的位混淆 (Bit Fiddling) 运算，软件应用最好使用素曲线；而对于硬件应用，最好使用二元曲线，它可以用非常少的门电路来实现快速且功能强大的 ECC 密码体制。本节主要讨论有限域上的椭圆曲线，第 5.4.3 节将讨论 $GF(2^m)$ 上构造的椭圆曲线。

对于有限域 Z_p 上的椭圆曲线，使用变元和系数均在 $0 \sim p-1$ 的整数集上取值的 3 次

方程, 其中 p 是大素数, 所执行的计算均是模 p 运算。与关于实数时的情形一样, 限制方程具有式 (5-23) 的形式, 但此处系数和变元均限制在 Z_p 中:

$$y^2 \bmod p = (x^3 + ax + b) \bmod p \quad (5-25)$$

例 5-6 $a=1, b=1, x=9, y=7, p=23$ 时可满足式 (5-24):

$$7^2 \bmod 23 = (9^3 + 9 + 1) \bmod 23$$

$$49 \bmod 23 = 739 \bmod 23$$

$$3 = 3$$

下面考虑所有满足式 (5-25) 的整数对 (x, y) 和无穷远点 O 组成的集合 $E_p(a, b)$ 。

例 5-7 取 $p=23$ 。考虑椭圆曲线方程 $y^2 = x^3 + x + 1$, 这里 $a = b = 1$ 。

注意, 该方程与图 5-1 (b) 中的方程是相同的。对 $E_{23}(1, 1)$, 只关心满足模 p 方程的, 从 $(0, 0)$ 到 $(p-1, p-1)$ 的象限中的非负整数。表 5-4 中列出了若干点 (除了元点 O 之外), 这些点是 $E_{23}(1, 1)$ 的一部分。

表 5-4 椭圆曲线 $E_{23}(1, 1)$ 上的点

(0, 1)	(6, 4)	(12, 19)
(0, 22)	(6, 19)	(13, 7)
(1, 7)	(7, 11)	(13, 16)
(1, 16)	(7, 12)	(17, 3)
(3, 10)	(9, 7)	(17, 20)
(3, 13)	(9, 16)	(18, 3)
(4, 0)	(11, 3)	(18, 20)
(5, 4)	(11, 20)	(19, 5)
(5, 19)	(12, 4)	(19, 18)

可以证明, 若 $(x^3 + ax + b) \bmod p$ 无重复因子, 则基于集合 $E_p(a, b)$ 可以定义一个有限 Abel 群。这等价于下列条件:

$$(4a^3 + 27b^2) \bmod p \neq 0 \bmod p \quad (5-26)$$

注意: 式 (5-26) 和式 (5-24) 具有相同的形式。

$E_p(a, b)$ 上的加法运算构造与定义在实数上的椭圆曲线中描述的代数方法是一致的。对任何点 $P, Q \in E_p(a, b)$, 有:

(1) $P + O = P$ 。

(2) 若 $P = (x_P, y_P)$, 则 $P + (x_P, -y_P) = O$ 。点 $(x_P, -y_P)$ 是 P 的负元, 记为 $-P$ 。

例如, 对于 $E_{23}(1, 1)$ 上的点 $P = (13, 7)$, 有 $-P = (13, -7)$ 。而 $-7 \bmod 23 = 16$, 因此, $-P = (13, 16)$, 该点也在 $E_{23}(1, 1)$ 上。

(3) 若 $P = (x_P, y_P)$, $Q = (x_Q, y_Q)$, 且 $P \neq -Q$, 则 $R = P + Q = (x_R, y_R)$ 由下列规则确定:

$$x_R = (\lambda^2 - x_P - x_Q) \bmod p$$

$$y_R = (\lambda(x_P - x_R) - y_P) \bmod p$$

其中

$$\lambda = \begin{cases} \left(\frac{y_Q - y_P}{x_Q - x_P} \right) \bmod p & \text{若 } P \neq Q \\ \left(\frac{3x_P^2 + a}{2y_P} \right) \bmod p & \text{若 } P = Q \end{cases}$$

(4) 乘法定义为重复相加。如 $4P = P + P + P + P$ 。例如，取 $E_{23}(1, 1)$ 上的 $P = (3, 10)$, $Q = (9, 7)$ ，那么

$$\lambda = \left(\frac{7-10}{9-3} \right) \bmod 23 = \left(\frac{-3}{6} \right) \bmod 23 = \left(\frac{-1}{2} \right) \bmod 23 = 11$$

$$x_R = (11^2 - 3 - 9) \bmod 23 = 109 \bmod 23 = 17$$

$$y_R = (11(3-17) - 10) \bmod 23 = -164 \bmod 23 = 20$$

所以 $P + Q = (17, 20)$ 。为计算 $2P$ ，首先求

$$\lambda = \left(\frac{3(3^2) + 1}{2 \times 10} \right) \bmod 23 = \left(\frac{5}{20} \right) \bmod 23 = \left(\frac{1}{4} \right) \bmod 23 = 6$$

上面等式的最后一步需要求 4 在 Z_{23} 中的乘法逆元。

$$x_R = (6^2 - 3 - 3) \bmod 23 = 30 \bmod 23 = 7$$

$$y_R = (6(3-7) - 10) \bmod 23 = (-34) \bmod 23 = 12$$

可见 $2P = (7, 12)$ 。

为了确定各种椭圆曲线密码的安全性，需要知道定义在椭圆曲线上的有限 Abel 群中点的个数。在有限群 $E_p(a, b)$ 中，点的个数 N 的范围是：

$$p + 1 - 2\sqrt{p} \leq N \leq p + 1 + 2\sqrt{p}$$

所以，对于大数 p ， $E_p(a, b)$ 上点的个数约等于 Z_p 中元素的个数。

5.4.3 $GF(2^m)$ 上的椭圆曲线

有限域 $GF(2^m)$ 由 2^m 个元素及定义在多项式上的加法和乘法运算组成。给定 m ，对 $GF(2^m)$ 上的椭圆曲线，可以使用变元和系数均在 $GF(2^m)$ 上取值的 3 次方程，且利用 $GF(2^m)$ 中的算术运算规则来进行计算。

可以证明， $GF(2^m)$ 上适合用于椭圆曲线密码的 3 次方程与 Z_p 上的 3 次方程有所不同，其形式如下：

$$y^2 + xy = x^3 + ax^2 + b \quad (5-27)$$

其中，变元 x 和 y 以及系数 a 和 b 是 $GF(2^m)$ 中的元素，且所有计算均在 $GF(2^m)$ 中进行。

考虑由满足式 (5-27) 的所有整数对 (x, y) 和无穷远点组成的集合 $E_{2^m}(a, b)$ 。可以证明，只要 $b \neq 0$ ，则可基于集合 $E_{2^m}(a, b)$ 定义一个有限 Abel 群。加法的运算规则如下：

对所有点 $P, Q \in E_{2^m}(a, b)$

(1) $P + O = P$ 。

(2) 若 $P = (x_p, y_p)$, 则 $P + (x_p, x_p + y_p) = O$ 。点 $(x_p, x_p + y_p)$ 是 P 的负元, 记为 $-P$ 。

(3) 若 $P = (x_p, y_p)$, $Q = (x_q, y_q)$, 且 $P \neq Q$, $P \neq -Q$, 则 $R = P + Q = (x_R, y_R)$ 由以下规则来确定:

$$\begin{aligned}x_R &= \lambda^2 + \lambda + x_p + x_q + a \\y_R &= \lambda(x_p + x_R) + x_R + y_p\end{aligned}$$

其中,

$$\lambda = \left(\frac{y_q + y_p}{x_q + x_p} \right)$$

(4) 若 $P = (x_p, y_p)$, 则 $R = 2P = (x_R, y_R)$ 由下列规则确定:

$$\begin{aligned}x_R &= \lambda^2 + \lambda + a \\y_R &= x_p^2 + (\lambda + 1)x_R\end{aligned}$$

其中,

$$\lambda = x_p + \frac{y_p}{x_p}$$

5.4.4 椭圆曲线密码

将 ECC 中的加密算法运算与 RSA 中的模乘运算相对应, 将 ECC 中的乘法运算与 RSA 中的模幂运算相对应。要建立基于椭圆曲线的密码体制, 则需要类似大合数分解或求离散对数这样的“数学难题”。

考虑方程 $Q = kP$, 其中 $Q, P \in E_p(a, b)$ 且 $k < p$ 。对于给定的 k 和 P 计算 Q 比较容易, 而对给定的 Q 和 P , 计算 k 则比较困难。

例 5-8 由方程 $y^2 \bmod 23 = (x^3 + 9x + 17) \bmod 23$ 所定义的群 $E_{23}(9, 17)$ 。

以 $P = (16, 5)$ 为底的 $Q = (4, 5)$ 的离散对数 k 是多少? 穷举攻击方法通过计算 P 的倍数来寻找 Q 。这样:

$P = (16, 5)$; $2P = (20, 20)$; $3P = (14, 14)$; $4P = (19, 20)$; $5P = (13, 10)$; $6P = (7, 3)$; $7P = (8, 7)$; $8P = (12, 17)$; $9P = (4, 5)$

因为 $9P = (4, 5) = Q$, 故以 $P = (16, 5)$ 为底的 $Q = (4, 5)$ 的离散对数 $k = 9$ 。在实际应用中 k 的值非常大, 从而使穷举攻击方法不可行。

在一些文献中, 分析了几种用椭圆曲线实现加/解密的方法。本节将介绍一种最简单的方法。

首先必须把要发送的消息明文 m 编码成形式为 (x, y) 的点 P_m , 并对点 P_m 进行加密, 然后对密文进行解密。注意, 不能简单地将消息编码成点的 x 坐标或 y 坐标, 因为并不是所有的坐标都在 $E_q(a, b)$ 中, 如表 5-4 所示。将消息编码成点 P_m 的方法有多种, 这里不讨论这些方法。但需要说明的是, 确实存在比较直接的编码方法。

首先, 挑选一个大的整数 q 以及式 (5-25) 或式 (5-27) 中的椭圆曲线参数 a 和 b , 这里 q 为素数 p 或是形为 2^m 的整数。由此可以定义出点的椭圆群 $E_q(a, b)$; 其次, 在 $E_q(a, b)$ 中挑选基点 $G = (x_1, y_1)$, G 的阶为一个非常大的数 n 。椭圆曲线上点 G 的阶 n 是

使得 $nG = O$ 成立的最小整数。

每个用户 A 选择一个私钥 n_A ，并产生公钥 $P_A = n_A \times G$ 。

若 A 要将消息 P_m 加密后发送给 B ，则 A 随机选择一个正整数 k ，并产生密文 C_m ，该密文是一个点对：

$$c_m = \{kG, P_m + kP_B\}$$

注意，此处使用了用户 B 的公钥 P_B 。

若 B 要对密文解密，则需要用第二个点减去第一个点与 B 的私钥之积：

$$P_m + kP_B - n_B(kG) = P_m + k(n_B G) - n_B(kG) = P_m$$

从上面可以发现， A 通过将 kP_B 与 P_m 相加来伪装消息 P_m ，因为只有 A 知道 k ，所以即使 P_B 是公钥，除了 A 之外，任何人均不能除去伪装。攻击者想要恢复明文消息，就必须通过 G 和 kG 求出 k ，但这被认为是非常困难的。

下面举例说明椭圆曲线的加密过程。取 $p = 751$ ， $E_p(-1, 188)$ ，即其椭圆曲线方程为 $y^2 = x^3 - x + 188$ ， $G = (0, 376)$ 。假定 A 要将已经编码成为椭圆曲线上的点 $P_m = (562, 201)$ 的消息发送给 B ，且 A 挑选随机数 $k = 386$ ， B 的公钥 $P_B = (201, 5)$ ，那么有

$$kG = 386 \times (0, 376) = (676, 558)$$

$$P_m + kP_B = (562, 201) + 386 \times (201, 5) = (385, 328)$$

于是， A 发送的密文是 $\{(676, 558), (385, 328)\}$ 。

5.4.5 椭圆曲线的安全性

ECC 的安全性建立在由 kP 和 P 确定 k 的困难程度之上，这个问题称为椭圆曲线的离散对数问题。Pollard Rho 方法是已知求椭圆曲线对数的最快的方法。表 5-5 对这种方法和分解两个素数之积的一般数域筛法进行了比较。由表 5-5 可知，ECC 使用的密钥比 RSA 中使用的密钥要短得多，而且在密钥长度相同时，ECC 与 RSA 所执行的计算量也差不多 [Jurisic 等 1997]。因此，与具有同等安全性的 RSA 相比，由于 ECC 使用更短的密钥，所以 ECC 所需的计算量比 RSA 少。

表 5-5 椭圆曲线密码和 RSA 在计算量上的比较

用 Pollard rho 方法求椭圆曲线对数		使用一般数域筛法进行整数因子分解	
密钥长度 (b)	MIPS 年	密钥长度 (b)	MIPS 年
—	—	512	3×10^4
—	—	768	2×10^8
150	3.8×10^{10}	1024	3×10^{11}
—	—	1280	1×10^{14}
205	7.1×10^{18}	1536	3×10^{16}
234	1.6×10^{28}	2048	3×10^{20}

5.4.6 ECC 的实现

美国 NeXT Computer 公司已开发出快速椭圆加密（FEE）算法，其密钥为容易记忆的字串。加拿大 Certicom 公司也开发出可实用的椭圆曲线密码体制（ECC）的集成电路（155b 和 12 000 个门的器件）[Certicom 1996]。该电路可实现高效加密、数字签名、认证和密钥管理等。Certicom 公司开发的产品包括：①CARDSECRETS，为 PC 卡信息安全模块；②FAXSECRETS，是独立应用的安全传真模块；③M*BIUS 可集入 Internet 或 PNTS 访问控制的安全解决方案。日本的 Mitsushita 公司、法国的 Thompson 公司、德国的 Siemens 公司和加拿大 Waterloo 大学等也都在实现这一体制。随着大整数分解和并行处理技术的进展，当前采用的公钥体制必须进一步增长密钥，这将使其速度更慢、更加复杂。ECC 则可用较小的开销（所需的计算量、存储量、带宽、软件和硬件实现的规模等）和时延（加密和签名速度高）实现较高的安全性，特别适用于计算能力和集成电路空间受限（如 PC 卡）、带宽受限（如无线通信和某些计算机网络），以及要求高速实现的情况。

Certicom 公司对 ECC 和 RSA 进行了对比，在实现相同的安全性下，ECC 所需的密钥量比 RSA 少得多，如表 5-6 所示。其中“MIPS 年”表示用每秒完成 100 万条指令的计算机所需工作的年数， m 表示 ECC 的密钥由 $2m$ 点构成。以 40MHz 的钟频实现 155b 的 ECC，每秒可完成 40 000 次椭圆曲线运算，其速度比 1024b 的 DSA 和 RSA 快 10 倍。

表 5-6 ECC 和 RSA 的对比

ECC 的密钥长度 m	RSA 的密钥长度	MIPS 年
160	1 024	1 012
320	5 120	1 036
600	21 000	1 078
1 200	120 000	10 168

ECC 特别适用于如下情况：

- （1）无线 Modem 的实现。对分组交换数据网提供加密，在移动通信器件上运行 4MHz 的 68330 CPU，ECC 可实现快速 Diffie-Hellman 密钥交换，并使密钥交换占用的带宽极小化，将计算时间从大于 60s 降到 2s 以下。
- （2）Web 服务器的实现。在 Web 服务器上集中进行密码计算会形成瓶颈，Web 服务器上的带宽有限使带宽费用高。采用 ECC 可节省计算时间和带宽，且通过算法的协商更易于处理兼容性。
- （3）集成电路卡的实现。ECC 无须协处理器就可以在标准卡上实现快速、安全的数字签名，这是 RSA 体制难以做到的。ECC 可使程序代码、密钥、证书的存储空间极小化，数据帧最短，便于实现，大大降低了 IC 卡的成本。

5.4.7 当前 ECC 的标准化工作

IEEE、ISO 和 ANSI 等标准化组织正在着手制定有关标准[Certicom 1996; Menezes

等 1996]。

1. IEEE P1363

椭圆曲线体制已被纳入 IEEE 公钥密码标准 P1363，其中包括加密、签名、密钥协议机制等。该标准完全支持 Z_p 和 F_{2m} 上的椭圆曲线体制。对于 F_{2m} 情况，它支持任意子域 F_{2l} 上 F_{2m} 的多项式基和正规基。标准 P1363 中也确定了离散对数（素数模下整数乘群子群中的）和 RSA 的加密和签名。其最新的草案可从 Web 地址 <http://stdsbds.ieee.org/groups/1363/index.html> 得到。

2. ANSI X9

椭圆曲线数字签名算法（ECDSA）标准 ANSI X9.62 是 X9F1 工作组提出的一个草案。ECDSA 给出一种采用椭圆曲线实现的数字签名算法，它类似于 NIST 的数字签名算法。ANSI X9.63 是由 X9F1 中的一个新的工作小组提出的椭圆曲线密钥协商和传输协议标准。它提出了几种采用椭圆曲线实现的密钥协商和密钥传输的方法。

3. ISO/IEC

《有后缀的数字签名（Digital Signature with Appendix）》CD 14888-3 给出对任意长的消息实现有后缀椭圆曲线数字签名算法，它类似于 ElGamal，特别类似于 DSA 签名算法。

4. AISO/IEC

互联网工程任务组（Internet Engineering Task Force, IETF）提出的密钥确定协议 OAKLEY KEY 是一种密钥协商协议，它类似于 Diffie-Hellman 协议。不同的组，包括 F2155h 和 F2210 上的椭圆曲线，都可以采用。该协议草案稿可从 Web 地址 <http://www.ietf.cnri.reston.va.us/> 得到。

5. ATM

异步传输模式（ATM）论坛技术委员会提出的 ATM 的安全性规范草案提出了 ATM 网的安全机制所提供的安全业务包括机密性、认证性、数据的完整性和访问控制。它支持各种体制，包括对称体制（如 DES）、非对称体制（如 RSA）和椭圆曲线体制。

5.4.8 椭圆曲线上的 RSA 密码体制

有文献[Koyama 等 1991]曾提出利用 Z_n 上的一类特殊的椭圆曲线构造类似于 RSA 的密码体制。Demytko 也提出类似方案。Vanstone 和 Zuccherato[Vanstone 等 1997]提出另一种方案。有关这类方案的安全性分析，可参阅相关文献[Kurosawa 等 1994; Kaliski 1997]。

5.4.9 用圆锥曲线构造双钥密码体制

有人提出用圆锥曲线构造双钥密码体制，但由于圆锥曲线是二次的，已证明存在亚指数分解算法，其上求离散对数的困难程度等价于 F_p 上的离散对数。

用超椭圆曲线构造双钥体制方法可参阅相关文献[Koblitz 1989; Adleman 等 1994; Shizuya 等 1991]。

ElGamal 算法是基于 $GF(2^m)$ 中乘群上定义的离散对数。这一算法不难推广到任意群

G 中的子群 H 上定义的离散对数。如果在 H 中的离散对数问题是困难问题, 则可将 ElGamal 体制推广到子群 H 上, 其中 $g \in G$, 且 $H = \{g^i, i \geq 0\}$, 明文集 $M = G$, 密文集 $C = G \times G$, 随机数 $k \in \mathbb{Z}_H$, 其他与 ElGamal 体制一样。特别强调的是, 在有限域上椭圆曲线 E 的点集所构成的群 G 上, 也可定义离散对数。当所用参数足够大时, 求逆在计算上是不可行的。这就为构造双钥密码体制提供了新的途径。

在此基础上构造的 ElGamal 密码体制, 其数据展宽系数为 4, 另外在椭圆曲线 E 上产生所需的点还没有方便的方法。在安全性方面, Menezes, Okamoto 和 Vanstone [Menezes 等 1991] 指出应避免选用超奇异 (Supersingular) 曲线, 否则椭圆曲线群上的离散对数问题退化为有限域低次扩域上的离散对数问题, 从而能在多项式时间上可解。他们还指出, 若所用循环子群的阶数达 2^{160} , 则可提供足够的安全性。

Menezes 和 Vanstone 曾提出另一种有效的方法: 以椭圆曲线作为“掩蔽”, 明文和密文可以是域中 (而不一定要求为 E 上的点) 任意非 0 有序域元素。这和原来的 ElGamal 密码体制一样, 因而这一体制的数据扩展系数为 2 [Menezes 1993; Okamoto 等 1994; Menezes 等 1993b]。

Buchman 和 William 提出一种用虚二次数域群构造公钥密码, 但在文献 [McCurley 1990] 提出亚指数时间计算离散对数算法后已无实用价值。

5.5 基于身份的密码体制

5.5.1 引言

1984 年, Shamir 提出了一种基于身份的加密方案 (Identity-Based Encryption, IBE) 的思想, 并征询具体的实现方案, 方案中不使用任何证书, 直接将用户的身份作为公钥, 以此来简化公钥基础设施 (Public Key Infrastructure, PKI) 中基于证书的密钥管理过程。例如用户 A 给用户 B 发加密的电子邮件, B 的邮件地址是 bob@company.com, A 只要将 bob@company.com 作为 B 的公开钥来加密邮件即可。当 Bob 收到加密的邮件后, 他与一个第三方——密钥服务器联系, 和向 CA 证明自己身份一样, B 向服务器证明自己, 并从服务器获得解密用的秘密钥, 再解密就可以阅读邮件。该过程如图 5-2 所示。



图 5-2 基于身份的加密方案示例

与现有的安全电子邮件相比, 即使 B 还未建立他的公钥证书, A 也可以向他发送加密的邮件。因此这种方法避免了公钥密码体制中公钥证书从生成、签发、存储、维护、更新、撤销这一复杂的生命周期过程。自 Shamir 提出这种新思想后, 由于没有找到有效的实现工具, 其实现一直是一个公开的问题。直到 2001 年, Dan Boneh 和 Matt Franklin 获得了数学上的突破, 提出了第一个实用的基于身份的公钥加密方案。他们的方案使用

椭圆曲线上的双线性映射（称为 Weil 配对和 Tate 配对），将用户的身份映射为一对公开钥/秘密钥对。双线性映射是满足 $\text{Pair}(aX, bY) = \text{Pair}(bX, aY)$ 的映射 Pair ，其中 a 和 b 是整数， X 和 Y 是椭圆曲线上的点。方案由 4 步组成，简单描述如下：

（1）初始化

密钥服务器选取一条椭圆曲线、秘密整数 s 、椭圆曲线上的点 P ，公开 P 和 sP 。

（2）加密

发送方 A 想向接收方 B 发送消息 M ，首先将 B 的身份（如 `bob@company.com`）经杂凑函数映射到椭圆曲线上的一个点，记为 Q_{ID} ，然后取一秘密的随机数 r ，计算 $k = \text{Pair}(rQ_{\text{ID}}, sP)$ ，作为加密密钥。最后将加密结果 $E_k(M)$ 和 rP 发给接收方 B 。其中 E 是一单钥加密算法。

（3）密钥产生

接收方 B 收到 $E_k(M)$ 和 rP 后，向密钥服务器提出申请，服务器在对 B 认证后，计算 sQ_{ID} 并发送给 B ， B 以 sQ_{ID} 作为密钥。

（4）解密

B 收到密钥后，计算 $k = \text{Pair}(sQ_{\text{ID}}, rP)$ ，使用 k 及 E 对密文解密。由于映射 Pair 的性质， B 计算的 k 与 A 使用的 k 相等。其他人不知道密钥 sQ_{ID} ，所以无法得到 k 。

5.5.2 双线性映射和双线性 D-H 假设

本节将用 Z_q 代表在 $\text{mod } q$ 加法下的群 $\{0, 1, \dots, q-1\}$ 。对于阶为素数的群 G ，用 G^* 代表集合 $G - \{O\}$ ，这里 O 为 G 中的单位元素。用 Z^+ 代表正整数集。

1. 双线性映射

设 q 是一大素数， G_1 和 G_2 是两个阶为 q 的群，其上的运算分别称为加法和乘法。 G_1 到 G_2 的双线性映射 $e: G_1 \times G_1 \rightarrow G_2$ ，满足下面的性质：

① 双线性。如果对任意 $P, Q, R \in G_1$ 和 $a, b \in Z$ ，有 $e(aP, bQ) = e(P, Q)^{ab}$ ，或 $e(P + Q, R) = e(P, R) \cdot e(Q, R)$ 和 $e(P, Q + R) = e(P, Q) \cdot e(P, R)$ ，那么就称该映射为双线性映射。

② 非退化性。映射不把 $G_1 \times G_1$ 中的所有元素对（即序偶）映射到 G_2 中的单位元。由于 G_1 、 G_2 都是阶为素数的群，这意味着：如果 P 是 G_1 的生成元，那么 $e(P, P)$ 就是 G_2 的生成元。

③ 可计算性。对任意的 $P, Q \in G_1$ ，存在一个有效算法计算 $e(P, Q)$ 。

Weil 配对和 Tate 配对是满足上述 3 条性质的双线性映射。

2. MOV 规约

G_1 中的离散对数问题是指已知 $P, Q \in G_1$ ，求 $\alpha \in Z_q$ ，使得 $Q = \alpha P$ 。已知这是一个困难问题，然而如果记 $g = e(P, P)$ ， $h = e(Q, P)$ ，则由 e 的双线性可知 $h = g^\alpha$ ，因此，可以将 G_1 中的离散对数问题归结为 G_2 中的离散对数问题，若 G_2 中的离散对数问题可解，则 G_1 中的离散对数问题可解。MOV 规约（也称 MOV 攻击）是指将攻击 G_1 中的离散对数问题转变为攻击 G_2 中的离散对数问题。所以要使 G_1 中的离散对数问题为困难问题，那么必

须选择适当参数使 G_2 中的离散对数问题为困难问题。

3. DDH 问题

G_1 中的判定性 Diffie-Hellman 问题简称 DDH (Decision Diffie-Hellman) 问题, 是指已知 P, aP, bP, cP , 判定 $c = ab \bmod q$ 是否成立, 其中 P 是 G_1^* 中的随机元素, a, b, c 是 Z_q^* 中的随机数。

由双线性映射的性质可知:

$$c = ab \bmod q \Leftrightarrow e(P, cP) = e(aP, bP)$$

因此可将判定 $c = ab \bmod q$ 是否成立转变为判定 $e(P, cP) = e(aP, bP)$ 是否成立, 所以 G_1 中的 DDH 问题是简单的。

4. CDH 问题

G_1 中的计算性 Diffie-Hellman 问题简称 CDH (Computational Diffie-Hellman) 问题, 是指已知 P, aP, bP , 求 abP , 其中 P 是 G_1^* 中的随机元素, a, b 是 Z_q^* 中的随机数。

与 G_1 中的 DDH 问题不同, G_1 中的 CDH 问题不因引入双线性映射而解决, 因此它仍是困难问题。

5. BDH 问题和 BDH 假设

由于 G_1 中的 DDH 问题简单, 那么就不能用它来构造 G_1 中的密码体制。IBE 体制的安全性是基于 CDH 问题的一种变形, 称为双线性 DH 假设。

双线性 DH 问题简称为 BDH (Bilinear Diffie-Hellman) 问题, 是指给定 $(P, aP, bP, cP) (a, b, c \in Z_q^*)$, 计算 $w = e(P, P)^{abc} \in G_2$, 其中 e 是一个双线性映射, P 是 G_1 的生成元, G_1, G_2 是阶为素数 q 的两个群。设算法 A 用来解决 BDH 问题, 其优势定义为 τ , 如果

$$\Pr | A(P, aP, bP, cP) = e(P, P)^{abc} | \geq \tau$$

目前还没有有效的算法解决 BDH 问题, 因此可假设 BDH 问题是一个困难问题, 这就是 BDH 假设。

5.5.3 IBE 方案

令 k 是安全参数, g 是 BDH 参数生成算法, 其输出包括素数 q , 两个阶为 q 的群 G_1, G_2 , 一个双线性映射 $e: G_1 \times G_1 \rightarrow G_2$ 的描述。 k 用来确定 q 的大小, 例如可以取 q 为 kb 长。

(1) 初始化。

给定安全参数 $k \in Z^+$, 算法运行如下:

- ① 输入 k 后运行 g , 产生素数 q , 两个阶为 q 的群 G_1, G_2 , 一个双线性映射 $e: G_1 \times G_1 \rightarrow G_2$ 。选择一个随机生成元 $P \in G_1$ 。
- ② 随机选取一个 $s \in Z_q^*$, 确定 $P_{\text{pub}} = sP$ 。
- ③ 选取一杂凑函数 $H_1: \{0, 1\}^* \rightarrow G_1^*$ 。对某个 n , 再选一个杂凑函数 $H_2: G_2 \rightarrow \{0, 1\}^n$ 。

安全分析时则把 H_1, H_2 视为随机语言*。

消息空间为 $\mathcal{M} = \{0,1\}^n$ ，密文空间为 $\mathcal{C} = G_1^* \times \{0,1\}^n$ 。系统参数为 $\langle q, G_1, G_2, e, n, P, P_{\text{pub}}, H_1, H_2 \rangle$ ，是公开的。 s 为主密钥，是保密的。

(2) 加密。

用接收方的身份 ID 作为公钥加密消息 $M \in \mathcal{M}$ ，需要 3 步：

① 计算 $Q_{\text{ID}} = H_1(\text{ID}) \in G_1^*$ 。

② 选择一个随机数 $r \in Z_q^*$ 。

③ 确定密文 $C = \langle rP, M \oplus H_2(g_{\text{ID}}^r) \rangle$ ，这里 $g_{\text{ID}} = e(Q_{\text{ID}}, P_{\text{pub}}) \in G_2^*$ ， $\oplus$ 是异或运算。

(3) 密钥产生。

对于一个给定的比特串 $\text{ID} \in \{0,1\}^*$ ，首先计算 $Q_{\text{ID}} = H_1(\text{ID}) \in G_1^*$ ，然后确定秘密钥 $d_{\text{ID}} = sQ_{\text{ID}}$ ，其中 s 为主密钥。

(4) 解密。

设密文为 $C = \langle U, V \rangle \in \mathcal{C}$ ，用秘密钥 d_{ID} 计算 $V \oplus H_2(e(d_{\text{ID}}, U)) = M$ 。

这是因为

$$e(d_{\text{ID}}, U) = e(sQ_{\text{ID}}, P)^s = e(Q_{\text{ID}}, P_{\text{pub}})^s = g_{\text{ID}}^s$$

杂凑函数有一个性质“对任一输入，其输出的概率分布与均匀分布在计算上是不可区分的”。若将这一性质改为“对任一输入，其输出是均匀分布的”，这样的杂凑函数是理想的。若把杂凑函数看做这样一个假想的理想函数，就称其为随机预言（Random Oracle）。

5.5.4 IBE 方案的安全性

1. 语义安全的基于身份的加密

公钥密码体制的语义安全的标准定义如下：

(1) 攻击算法已知一个由系统产生的随机公钥。

(2) 攻击算法输出两个长度相同的消息 M_0, M_1 ，再从系统接收 M_b 的密文，其中随机值 $b \in \{0,1\}$ 。

(3) 攻击算法输出 b' ，如果 $b = b'$ 则成功。如果没有多项式时间的攻击算法能以不可忽略的优势成功，那么该密码体制就是语义安全的。

要定义基于身份的密码体制的语义安全，应允许攻击算法根据自己的选择进行密钥询问，即攻击算法可根据自己的选择询问公钥 ID 对应的密钥，以此来加强标准定义。如果不存在多项式时间的攻击算法 A ，以不可忽略的优势在下面的攻击中获得成功，那么就称此方案是语义安全的。

(1) 初始化：系统输入安全参数 k ，产生公开的系统参数 Params 和保密的主密钥。

(2) 阶段 1：攻击算法发出对 $ID_1, \dots, ID_m$ 的密钥产生询问。系统允许密钥产生算法，产生与公钥 ID_i 对应的密钥 d_i ($i=1, \dots, m$)，并把它发送给攻击算法。

询问：攻击算法输出两个长度相等的明文 M_0, M_1 和一个意欲询问的公开钥 ID。唯一

的限制是 ID 不在阶段 1 中的任何密钥询问中出现。系统随机选取一个比特值 $b \in \{0,1\}$, 计算 $C = \text{Encrypt}(\text{Params}, \text{ID}, M_b)$, 并将 C 发送给攻击算法。

(3) 阶段 2: 攻击算法发出对 $\text{ID}_{m+1}, \dots, \text{ID}_n$ 的密钥产生询问, 唯一的限制是 $\text{ID}_i \neq \text{ID}$ ($i=m+1, \dots, n$), 系统以阶段 1 中的方式进行回应。

最后, 攻击算法输出猜测 $b' \in \{0,1\}$, 如果 $b = b'$ 则成功。

攻击算法的优势可定义为参数 k 的函数: $\text{Adv}_{\epsilon, A}(k) = \left| \Pr[b = b'] - \frac{1}{2} \right|$

定义 5-5 如果对于任何多项式时间的攻击算法, $\text{Adv}_{\epsilon, A}(k)$ 可忽略, 那么就称这个 IBE 体制是语义安全的。

一个函数 $g: R \rightarrow R$ 是可以忽略的, 意指对任意 $d > 0$ 和一个充分大的 k 有 $|g(k)| < 1/k^d$ 。

定理 5-1 设杂凑函数 H_1, H_2 是随机预言, 如果在 g 产生的群中 BDH 问题是困难的, 那么上述 IBE 方案是语义安全的基于身份的加密方案。

证明过程略。

2. 选择密文安全

选择密文安全是公钥加密方案的一个标准安全概念, 在 IBE 体制中这个要求需再加强些, 因为在 IBE 体制中, 攻击算法攻击公钥 ID (即获取与之对应的密钥) 时, 他可能已有所选用户 $\text{ID}_1, \dots, \text{ID}_n$ 的密钥, 因此选择密文安全的定义就应允许攻击算法获取与其所选身份 (但不是 ID) 相应的秘密钥, 可把这一要求看做对密钥产生算法的询问。

一个 IBE 加密方案是抗自适应选择密文攻击语义安全的, 如果不存在多项式时间的攻击算法, 它在下面的攻击过程中有不可忽略的概率。

(1) 初始化: 系统输入安全参数 k , 产生公开的系统参数 Params 和保密的主密钥。

(2) 阶段 1: 攻击算法执行 $q_1, q_2, \dots, q_m$, 这里 q_i 是下面的询问之一:

- 对 $\langle \text{ID}_i \rangle$ 的密钥产生询问。系统运行密钥产生算法, 产生与公钥 ID_i 对应的密钥 d_i , 并把它发送给攻击算法。
- 对 $\langle \text{ID}_i, C_i \rangle$ 的解密询问。系统运行密钥产生算法, 产生与 ID_i 对应的秘密钥 d_i , 再运行解密算法, 用 d_i 解密 C_i , 并将所得明文发送给攻击算法。

上面的询问可以自适应地进行, 是指执行每个 q_i 时可以依赖于执行 $q_1, q_2, \dots, q_{i-1}$ 时得到的询问结果。

攻击算法输出两个长度相等的明文 M_0, M_1 和一个要被询问的身份 ID。唯一的限制是 ID 不出现在阶段 1 中的任何密钥询问中。

系统选取一个随机值 $b \in \{0,1\}$, 产生 $C = \text{Encrypt}(\text{Params}, \text{ID}, M_b)$, 并将 C 作为应答发送给攻击算法。

(3) 阶段 2: 攻击算法产生更多询问 $q_{m+1}, \dots, q_n, q_i$ 是下面的询问之一:

- 对 $\langle \text{ID}_i \rangle$ 的密钥产生询问。系统以阶段 1 中的方式进行回应。
- 对 $\langle \text{ID}_i, C_i \rangle$ 的解密询问。系统以阶段 1 中的方式进行回应。

最后, 攻击算法输出对 b 的猜测 $b' \in \{0,1\}$, 如果 $b = b'$ 则成功。

以上攻击过程也称为“午餐时间攻击”或“午夜攻击”，相当于有一个执行解密运算的黑盒，掌握黑盒的人在午餐时间离开后，攻击者能使用黑盒对自己选择的密文解密。午餐过后，给攻击者一个目标密文，攻击者试图对目标密文解密，但不能再使用黑盒了。

定义

$$\text{Adv}_{\varepsilon, A}(k) = \left| \Pr[b = b'] - \frac{1}{2} \right|$$

为攻击算法的优势。

定义 5-6 如果对所有多项式时间的攻击算法，函数 $\text{Adv}_{\varepsilon, A}(k)$ 可忽略，那么就称该 IBE 体制是抗自适应性选择密文攻击语义安全的。

为使上述方案成为在随机预言模型中是选择密文安全的，还需对其加以修改。以 $\varepsilon_{\text{pk}}(M, r)$ 表示用随机比特 r 在公钥 pk 下加密 M 的公钥加密算法，Fujisaki-Okamoto 指出，如果 ε_{pk} 是单向加密的，则 $\varepsilon_{\text{pk}}^{\text{hy}} = \langle \varepsilon_{\text{pk}}(\sigma, H_3(\sigma, M)), H_4(\sigma) \oplus M \rangle$ 在随机预言模型下是选择密文安全的，其中 σ 是随机产生的比特串， H_3, H_4 是杂凑函数。

单向加密粗略地讲就是对于一个给定的随机密文，攻击算法无法产生明文。单向加密是一个弱安全概念，这是因为它没有阻止攻击算法获得明文的部分比特值。

修改后的加密方案如下：

(1) 初始化。

和基本方案相同，此外还需选取两个杂凑函数 $H_3: \{0,1\}^n \times \{0,1\}^n \rightarrow Z_q^*$ 和 $H_4: \{0,1\}^n \rightarrow \{0,1\}^n$ ，其中 n 是待加密消息的长度。

(2) 加密。

用公钥 ID 加密 $M \in \{0,1\}^n$ ：

① 计算 $Q_{\text{ID}} = H_1(\text{ID}) \in G_1^*$ 。

② 选一个随机串 $\sigma \in \{0,1\}^n$ 。

③ 计算 $r = H_3(\sigma, M)$ 。

④ 确定密文 $C = \langle rP, \sigma \oplus H_2(g_{\text{ID}}^r), M \oplus H_4(\sigma) \rangle$ ，这里 $g_{\text{ID}} = e(Q_{\text{ID}}, P_{\text{pub}}) \in G_2$ 。

(3) 密钥产生。

和基本方案相同。

(4) 解密。

令 $C = \langle U, V, W \rangle$ 是用 ID 加密所得的密文。如果 $U \notin G_1^*$ ，拒绝这个密文。否则，用秘密钥 $d_{\text{ID}} \in G_1^*$ 对 C 如下解密：

① 计算 $V \oplus H_2(e(d_{\text{ID}}, U)) = \sigma$ 。

② 计算 $W \oplus H_4(\sigma) = M$ 。

③ 确定 $r = H_3(\sigma, M)$ ，检验 $U = rP$ 是否成立，不成立则拒绝密文。

④ 把 M 作为 C 的明文。

定理 5-2 设杂凑函数 H_1, H_2, H_3, H_4 是随机预言，假设在由 g 生成的群中 BDH 问题是困难的，那么上述修改后的 IBE 是选择密文安全的。

5.6

中国商用密码 SM2 算法

2010 年 12 月 17 日, 国家密码管理局颁布了中国商用公钥密码标准算法 SM2。它是一组基于椭圆曲线的公钥密码算法。本节介绍 SM2 公钥加解密算法。SM2 数字签名算法将在第 7 章介绍。国家密码管理局公告(第 21 号)详细描述了 SM2 系列算法。读者可从 Web 地址 http://www.oscca.gov.cn/News/201012/News_1197.htm 得到算法详细描述。

5.6.1 SM2 椭圆曲线推荐参数

SM2 椭圆曲线系统参数如下:

- ① 有限域 F_q 的规模 q (当 $q = 2^m$ 时, 还包括元素表示法的标识和约化多项式)。
- ② 定义椭圆曲线 $E(F_q)$ 方程的两个元素 $a, b \in F_q$ 。
- ③ 椭圆曲线 $E(F_q)$ 上的基点 $G = (x_G, y_G)$ ($G \neq O$), 其中 x_G, y_G 是 F_q 的两个元素。
- ④ G 的阶 n 以及其他可选项, 如 n 的余因子 h 等。

SM2 椭圆曲线公钥密码算法推荐使用 256 位素数域 $GF(p)$ 上的椭圆曲线, 椭圆曲线方程描述为

$$y^2 = x^3 + ax + b \quad (5-28)$$

SM2 椭圆曲线推荐参数用十六进制表述为

$p =$	FFFFFFFFE FFFFFFFFF	FFFFFFFFF 00000000	FFFFFFFFF FFFFFFFFF	FFFFFFFFF FFFFFFFFF
$a =$	FFFFFFFFE FFFFFFFFF	FFFFFFFFF 00000000	FFFFFFFFF FFFFFFFFF	FFFFFFFFF FFFFFFFFC
$b =$	28E9FA9E F39789F5	9D9F5E34 15AB9F92	4D5A9E4B DDBCBD41	CF6509A7 4D940E93
$n =$	FFFFFFFFF 7203DF6B	FFFFFFFFF 21C6052B	FFFFFFFFF 53BBF409	FFFFFFFFF 39D54123

此椭圆曲线建议基点 G 为

$x_G =$	32C4AE2C 8FE30BBF	1F198119 F2660BE1	5F990446 715A4589	6A39C994 334C74C7
$y_G =$	BC3736A2 B0A9877C	F4F6779C C62A4740	59BDC EE3 02DF32E5	6B692153 2139F0A0

5.6.2 辅助函数

SM2 椭圆曲线公钥加解密算法涉及 3 类辅助函数: 杂凑函数、密钥派生函数和随机数发生器。这 3 类辅助函数的安全性强弱直接影响加密算法的安全性。因此, 实际使用

SM2 椭圆曲线公钥加解密算法时应使用标准中指定的辅助函数。

1. 杂凑函数

杂凑函数的作用是将任意长的数字串 M 映射成一个较短的定长输出数字串的函数, 一般用 H 表示。杂凑算法的详细内容将在第 6 章介绍。在 SM2 椭圆曲线公钥加解密算法中, 应使用国家密码管理局批准的杂凑算法, 如 SM3 杂凑算法。

2. 密钥派生函数

密钥派生函数的作用是从一个共享的秘密比特串中派生出密钥数据。本质上, 密钥派生函数是一个伪随机数产生函数, 用来产生所需的会话密钥或进一步加密所需的密钥数据。SM2 椭圆曲线公钥加解密算法中详细规定了基于杂凑函数的密钥派生函数。因此, 密钥派生函数需要调用杂凑函数。

密钥派生函数所调用的杂凑函数用 H_v 来描述, 其输出是长度恰好为 v 比特的杂凑值。密钥派生函数用 $K \leftarrow \text{KDF}(Z, \text{klen})$ 来描述, 其中 Z 是输入的比特串, klen 是要获得密钥数据的比特长度, 要求 klen 小于 $(2^{32}-1)v$ 。KDF 输出长度为 klen 的密钥数据比特串 K 。密钥派生函数的具体算法流程描述如下:

- ① 初始化一个 32 比特构成的计数器 $\text{ct}=0\text{x}00000001$ 。
- ② 对 i 从 1 到 $\lceil \text{klen}/v \rceil$: 计算 $Ha_i = H_v(Z \parallel \text{ct})$, 并令 ct 加 1。
- ③ 若 klen/v 非整数, 令 $Ha_{\lceil \text{klen}/v \rceil}$ 为 $Ha_{\lceil \text{klen}/v \rceil}$ 最左边的第 $(\text{klen} - (v \times \lceil \text{klen}/v \rceil))$ 比特。否则, 令 $Ha_{\lceil \text{klen}/v \rceil} = Ha_{\lceil \text{klen}/v \rceil}$ 。
- ④ 输出密钥数据比特串 $K = Ha_1 \parallel Ha_2 \parallel \dots \parallel Ha_{\lceil \text{klen}/v \rceil-1} \parallel Ha_{\lceil \text{klen}/v \rceil}$ 。

3. 随机数发生器

随机数发生器的作用是从指定的集合范围内产生随机数。随机数发生器必须满足随机性和不可预测性。在 SM2 椭圆曲线公钥加解密算法中, 应使用国家密码管理局批准的随机数发生器。

5.6.3 密钥生成

私钥: 用户的私钥为一个随机数 $d \in \{1, 2, \dots, n-1\}$ 。

公钥: 用户的公钥为椭圆曲线上的点 $P = dG$ 。

5.6.4 加密

设需要发送的消息为比特串 M , klen 为 M 的比特长度。为了对明文 M 进行加密, 作为加密者的用户 A 获得用户 B 的公钥 P_B 后, 应执行如下运算步骤:

- ① 用随机数发生器产生随机数 $k \in \{1, 2, \dots, n-1\}$ 。
- ② 计算椭圆曲线上的点 $C_1 = kG = (x_1, y_1)$, 并将 C_1 的数据类型转换为比特串。
- ③ 计算椭圆曲线上的点 $S = hP_B$, 若 S 是无穷远点, 则报错并退出。
- ④ 计算椭圆曲线上的点 $kP_B = (x_2, y_2)$, 并将坐标 x_2, y_2 的数据类型转换为比特串。

⑤ 计算 $t = \text{KDF}(x_2 \| y_2, \text{klen})$, 若 t 为全 0 比特串, 则返回①。

⑥ 计算 $C_2 = M \oplus t$ 。

⑦ 计算 $C_3 = \text{Hash}(x_2 \| M \| y_2)$ 。

⑧ 输出密文 $C = C_1 \| C_2 \| C_3$ 。

注意: 第⑦步所使用的杂凑函数 Hash 也应使用中国商用密码标准中的杂凑函数。

图 5-3 为 SM2 椭圆曲线公钥加密算法的流程图。

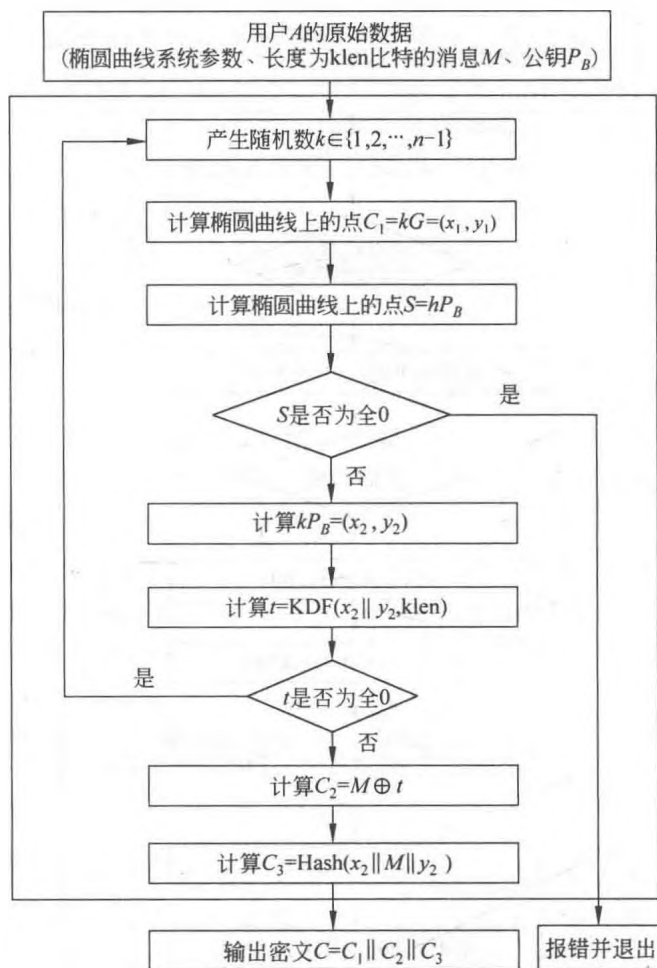


图 5-3 SM2 加密算法流程图

5.6.5 解密

设需要解密的密文为 $C = C_1 \| C_2 \| C_3$, klen 为密文中 C_2 的比特长度。为了对密文 C 进行解密, 作为解密者的用户 B 应用其私钥 d_B 执行如下运算步骤:

① 从 C 取出比特串 C_1 , 将 C_1 的数据类型转换为椭圆曲线上的点, 验证 C_1 是否满足椭圆曲线方程。若不满足, 则报错并退出。

② 计算椭圆曲线上的点 $S = h C_1$, 若 S 是无穷远点, 则报错并退出。

③ 计算 $d_B C_1 = (x_2, y_2)$, 并将坐标 x_2, y_2 的数据类型转换为比特串。

- ④ 计算 $t = \text{KDF}(x_2 \| y_2, \text{klen})$ ，若 t 为全 0 比特串，则返回①。
- ⑤ 从 C 取出比特串 C_2 ，计算 $M' = C_2 \oplus t$ 。
- ⑥ 计算 $u = \text{Hash}(x_2 \| M' \| y_2)$ ，从 C 取出比特串 C_3 ，若 $u \neq C_3$ ，则报错并退出。
- ⑦ 输出明文 M' 。

注意：第⑥步所使用的杂凑函数 Hash 应与加密函数第⑦步所使用的杂凑函数一致。

图 5-4 为 SM2 椭圆曲线公钥解密算法的流程图。

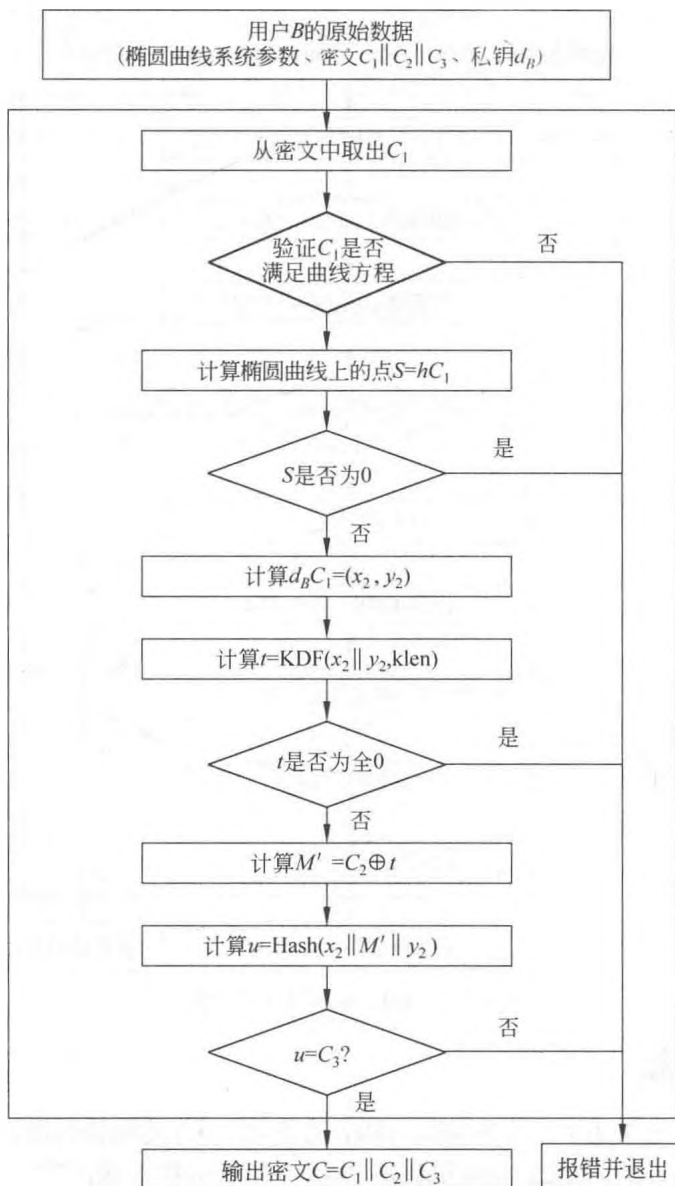


图 5-4 SM2 解密算法流程图

我们很容易证明加解密的正确性。由加密算法可知：

$$C_1 = kG = (x_1, y_1) \quad (5-29)$$

由公私钥关系、加密算法的第④步、解密算法的第③步可知：

$$d_B C_1 = d_B kG = kP_B = (x_2, y_2) \quad (5-30)$$

因此,解密算法第④步可得到正确的会话密钥 t ,经第⑤步得到正确的明文 $M'=M$ 。

5.6.6 实例与应用

为了开发人员在工程实现时调试方便,中国国家密码管理局在颁布 SM2 公钥加解密算法时,分别给出了 SM2 公钥加解密算法在 F_p -256 上椭圆曲线和在 F_{2^m} -257 上椭圆曲线的消息加解密实例,以及加密解密各步骤中的有关值。感兴趣的读者可从 Web 地址 http://www.oscca.gov.cn/News/201012/News_1197.htm 获取相关实例信息。

SM2 公钥加解密算法也属于 ElGamal 型椭圆曲线密码。但 SM2 公钥加解密算法加入了很多检错措施,提高了密码系统的数据完整性和可靠性。例如,解密算法第①步,通过验证 C_1 是否满足椭圆曲线方程来验证 C_1 的有效性;解密算法第②步,通过验证子群元素的阶进一步检查 C_1 的有效性;解密算法第⑥步,应用所解密的明文 M' 以及坐标值 x_2, y_2 检查 C_3 的正确性,而所解密明文 M' 的正确性包含 C_2 与 t 的正确性。因此,经过解密步骤①、②、⑥,密文 $C = C_1 \| C_2 \| C_3$ 的正确性与有效性均得到验证。

SM2 公钥密码学算法已在中国得到了广泛应用。在中华人民共和国居民身份证的芯片中就用硬件实现了 SM2 公钥加解密算法,用来保护重要的个人信息。截至 2013 年 8 月 31 日,共有 352 项通用产品支持 SM2 公钥加解密算法;截至 2016 年 2 月 29 日,共有 564 项商用密码产品支持 SM2 公钥加解密算法。感兴趣的读者可从 Web 网址 http://www.oscca.gov.cn/News/201109/News_1212.htm 获取支持 SM2 公钥加解密算法的通用产品名单;从 Web 网址 http://www.oscca.gov.cn/News/201603/News_1319.htm 获取支持 SM2 公钥加解密算法的商用密码产品目录。

5.7

公钥密码体制的安全性分析

通常说“密码体制 X 对于攻击 Y 是安全的,但是对于攻击 Z 是不安全的”是有道理的,即密码体制的安全性是根据攻击来定义的。主动攻击通常有 3 种方式,这些主动攻击的方式将用于对本章其余部分所介绍的密码体制的分析,它们的定义如下。

(1) 选择明文攻击 (CPA)。攻击者选择明文消息并得到加密服务,产生相应的密文。攻击者的任务是用所得到的明/密文对来降低目标密码体制的安全性。

(2) 选择密文攻击 (CCA)。攻击者选择密文消息并得到解密服务,产生相应的明文。攻击者的任务是用所得到的明/密文对来降低目标密码体制的安全性。在解密服务停止后,即在得到目标密文之后,解密服务立即停止。如果攻击者能够从“目标密文”中得到保密明文的信息,则就说攻击是成功的。

(3) 适应性选择密文攻击 (CCA2)。这是一个 CCA,而且除了对“目标密文”解密外,永远能够得到解密服务。

可以用以下情形来想象上述攻击类型:

(1) 在 CPA 中, 攻击者有一个加密盒子。

(2) 在 CCA 中, 攻击者可以有条件地使用解密盒子: 在交给攻击者目标密文之前关闭解密盒子。

(3) 在 CCA2 中, 在攻击者得到目标密文之前或之后, 只要攻击者不把目标密文输入解密盒子(这个唯一的限制是合理的, 否则攻击者就没有任何需要解决的困难问题了), 他就可以一直使用这个解密盒子。

在所有的情况下, 攻击者都不应该拥有相应的密钥。

CPA 和 CCA 原来是作为攻击对称密码系统所提出的主动密码分析模型, 在对称密码系统中, 攻击者的目标就是用他从攻击中得到的明/密文对减弱目标加密系统的安全性。它们已经用于规范对公钥系统的主动攻击。这里指出以下有关公钥密码系统的 3 个细节:

(1) 在公钥系统下, 由于给定了公钥, 任何人都可以完全控制加密算法, 这样任何人总是可以得到公钥系统的加密服务。换句话说, CPA 永远可以用来攻击公钥密码系统。于是, 如果对公钥密码系统的一个攻击没有用到任何解密服务, 就可以称这个攻击为 CPA。因此, 显然任何一个公钥密码系统必须抵抗 CPA, 否则它就不是一个有用的密码系统。

(2) 一般地, 大多数公钥密码体制基于的数学问题都有一些很好的代数结构性质, 如闭包、结合律和同态等。一个攻击者可以运用这些很好的性质, 并通过巧妙的计算组成一条密文。如果攻击者能得到解密服务, 则他的巧妙的计算可能使他得到一些明文信息, 或者甚至是目标加密系统的私钥, 否则要得到私钥对他来说在计算上是不可行的。所以, 公钥系统特别容易受到 CCA 和 CCA2 的攻击。

(3) 看起来 CCA 限制太大了。在应用中, 处于攻击下的用户(被要求提供解密服务)实际上未必知道攻击的存在。所以用户就不知道何时应该停止提供解密服务。一般假设普通用户不知道攻击者的存在, 所以攻击者一直能够得到解密服务。另一方面, 由于攻击者总能够自己来执行选择明文的加密“服务”, 所以任何公钥系统都必须抵抗 CPA。由于这个原因, 主要考虑抵抗 CCA2 的方法。

最近, 人们对抗选择密文攻击的双钥密码有不少研究。Goldwasser 等学者[Goldwasser 等 1988]最先指出并非所有双钥体制的解密问题都像从公钥恢复密钥一样困难, 因此必须注意双钥体制经受选择密文攻击的能力。Naor 和 Yung[Naor 等 1990]首次建议了一种抗冷漠选择密文攻击在语义上安全的具体公钥加密方案。此方案采用了两个独立的概率公钥加密方案对明文加密, 以后以非交互零知识证明方式送出。其中同一个消息采用两个密钥加密。Rackoff 和 Simon[Rackoff 等 1991]首次提出一种抗自适应选择密文攻击在语义上安全的公钥加密方案。但这类方案都由于消息扩展太大而不实用。

Damgård 也曾提出一种可以抗冷漠选择密文攻击的有效构造公钥体制的方法, Zheng 和 Seberry[Zheng 等 1993]指出, 该体制不能抗自适应选择明文攻击, 并提出 3 种方法对抗此类攻击。但这些方案都未能证明可以达到所宣称的安全水平。后来 Bellare 和 Gogaway[Bellare 等 1993]证明 Zheng 等提出的方案中的随机预言模型式(Random Oracle Model)在自适应选择密文攻击下是可证明安全的。Lim 和 Lee [Lim 等 1993]曾提出可

以抗选择密文攻击的公钥方案, 但被 Frankel 和 Yung[Frankel 等 1995]攻破。

习 题

一、填空题

1. 在双钥密码体制中, 若以_____作为加密密钥, 以_____作为解密密钥, 则可实现多个用户加密的消息只能由一个用户解读; 若以_____作为加密密钥, 以_____为解密密钥, 则可实现一个用户加密的消息能由多个用户解读。
2. 对于双钥密码体制来说, 其安全性主要取决于_____, 要求加密函数具有_____。
3. DL 问题是指已知 y, g, p , 求_____的问题; DHP 问题是指已知 α^a 和 α^b , 求_____的问题; FAC 问题是指已知 $n = p \parallel q$, 求_____和_____的问题。
4. 双钥密码体制需要基于单向函数来构造, 目前多数双钥体制是基于_____, _____, _____和_____等问题构造的。
5. RSA 密码体制易于实现, 既可用于_____又可用于_____, 是被广泛应用的一种公钥体制。
6. 针对 RSA 密码体制的选择明文攻击, 包括_____, _____和_____等方式。
7. ElGamal 密码体制是一种基于_____的双钥密码体制, 其加密密文是由明文和所选随机数 k 来确定, 因而属于_____加密。
8. 椭圆曲线密码体制利用有限域上的_____所构成的群上定义的_____构造双钥密码体制。
9. 基于身份的密码体制, 使用双椭圆曲线上的_____, 将用户的身份映射为_____。
10. 中国商用公钥密码标准算法 SM2 算法是一组基于_____的公钥密码算法。SM2 公钥加解密算法中包含 3 类辅助函数, 分别为_____, _____和_____。

二、思考题

1. 什么是单向函数? 什么是陷门单向函数?
2. 双钥体制的安全性均依赖于构造双钥算法所依赖的数学难题。那么 RSA 算法是基于一种什么数学难题构造的?
3. 离散对数问题与计算 Diffie-Hellman 问题有什么关系?
4. 在 RSA 公钥数据 (e, N) 中, 为什么加密指数 e 必须与 $\varphi(N)$ 互素?
5. 通常情况下分解奇合数是困难问题。那么分解素数的幂也是困难问题吗? (一个素数幂是 $N = p^i$, 其中 p 是素数, i 是整数。分解 N 。)(提示: 对任意 $i > 1$, 计算 N 的 i 次根需要尝试多少个指数值 i ?)
6. RSA 加密函数可以看做 RSA 模数乘群上的一个置换, 所以 RSA 函数也称为单向陷门置换。ElGamal 加密函数是单向陷门置换吗?

7. 在什么情况下可以把 ElGamal 密码体制看做是确定的算法？
8. 与 RSA 相比，ECC 的主要优点是什么？试将两者进行比较。
9. SM2 公钥加解密算法与椭圆曲线 ElGamal 公钥加解密算法相比，有什么相似之处？有什么不同之处？SM2 公钥加解密算法增加了何种功能？试将两者进行比较。
10. 什么是 CPA，CCA 和 CCA2？为什么所有公钥加密算法都必须抵抗 CPA？
11. 由于主动攻击通常要修改网络上传输的（密文）消息，那么如果公钥加密算法用了数据完整性检测技术来检测对密文消息的非授权修改，主动攻击仍然会有效吗？